Universidad de las Fuerzas Armadas ESPE

Departamento: Electrónica y Telecomunicaciones

Asignatura: Fundamentos de Programación

NRC: 36466

Apellidos y Nombres: Guacho Usuay César Ariel

Deber: Aplicación de CodeBlocks en **Ejercicios de programación y algoritmia.**

Problema 1.1. Media de dos valores

Desarrolla un programa que ingrese 2 números por teclado e imprima su media.

1. **Análisis del problema**

Entrada: 2 números reales, num1 y num2.

Proceso: Sumar ambos números y dividir el resultado entre 2.

Salida: Media aritmética de los 2 números.

2. **Tabla de objetos**

Objeto	Nombre	Valor	Tipo
Variable	num1	Ingresado por teclado	Real
Variable	num2	Ingresado por teclado	Real
Variable	media	(num1 + num2) / 2	Real

3. **Seudocódigo**

```
Inicio
  Definir num1, num2, media Como real

  Escribir "Ingrese el primer número:"
  Leer num1

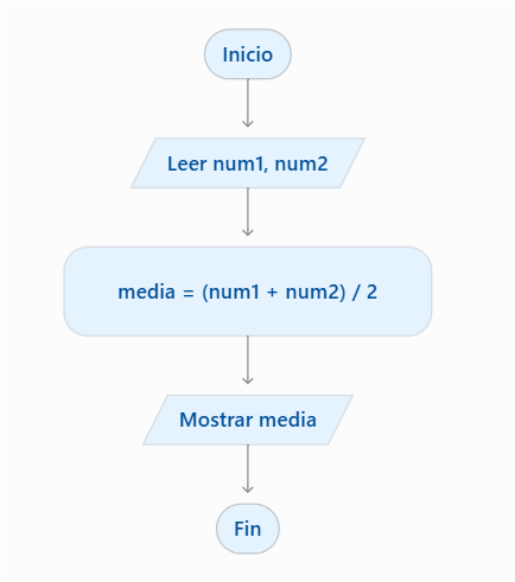
  Escribir "Ingrese el segundo número:"
  Leer num2

  media ← (num1 + num2) / 2
```

Escribir "La media es:", media

FinAlgoritmo

4. **Diagrama de Flujo**



5. **Prueba de escritorio**

num1	num2	Operación	Resultado
8	12	(8 + 12) / 2	10
15	25	(15 + 25) / 2	20
7	4	(7 + 4) / 2	5,5

6. **Código en C**

```
#include <stdio.h>

int main() {
   float num1, num2, media;

   printf("Ingrese el primer numero: ");
   scanf("%f", &num1);

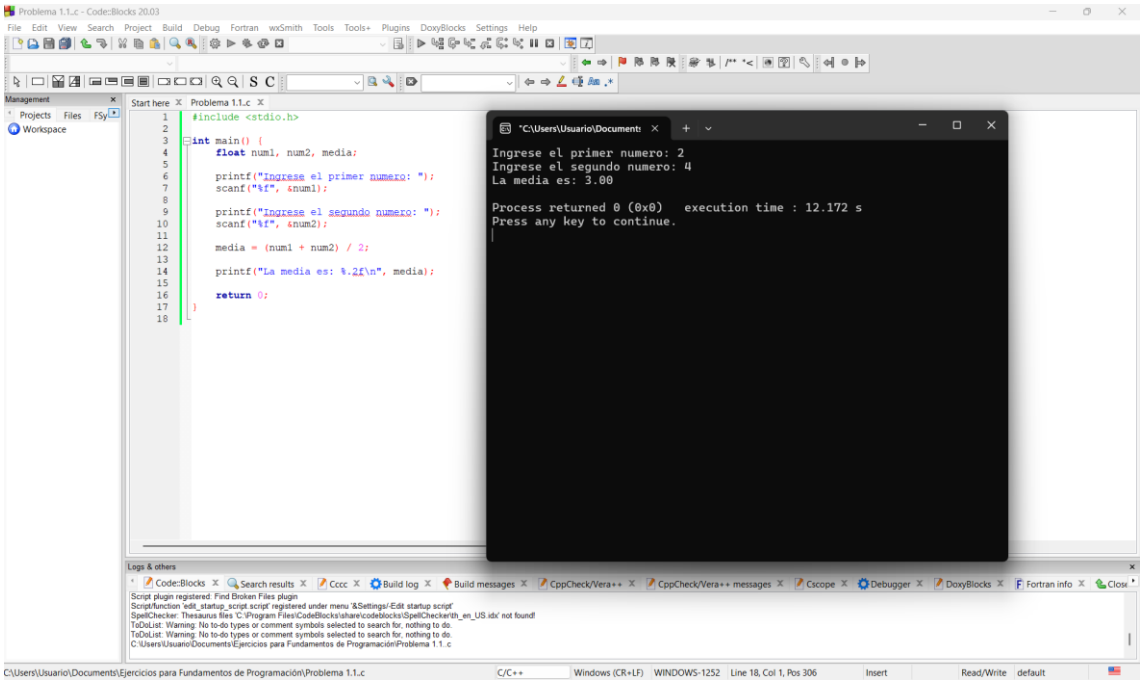
   printf("Ingrese el segundo numero: ");
   scanf("%f", &num2);

   media = (num1 + num2) / 2;
```

```
    printf("La media es: %.2f\n", media);


    return 0;
}
```

7. Evidencia



Problema 1.2. Valor absoluto de X al cubo

Desarrollar un programa que lea un número por teclado y escriba por pantalla |x|^3.

1. Análisis del problema

Entrada: Un número real X.

Proceso: Calcular el valor absoluto de X y elevarlo al cubo.

Salida: Resultado de |x|^3.

2. Tabla de objetos

Objeto	Nombre	Valor	Tipo
Variable	x	Ingresado por teclado	Real
Variable	absoluto	|x|	Real
Variable	resultado	absoluto ^3	Real

3. Seudocódigo

Inicio

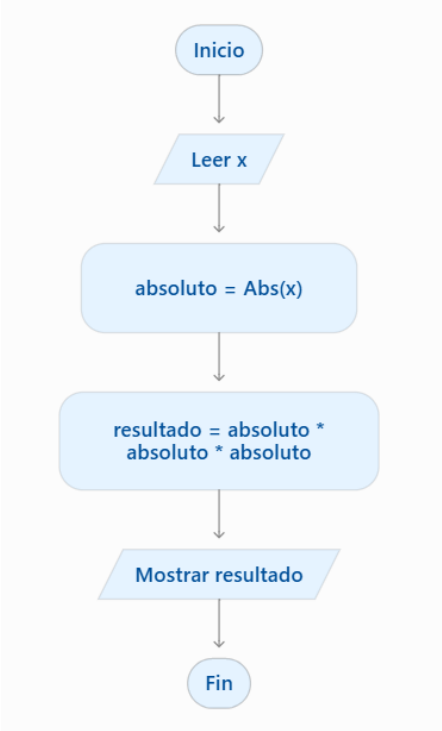
```
Definir x, absoluto, resultado Como Real

Escribir "Ingrese un número:"
Leer x

absoluto ← Abs(x)
resultado ← absoluto * absoluto * absoluto

Escribir "Resultado:", resultado
FinAlgoritmo
```

4. **Diagrama de flujo**



5. **Prueba de escritorio**

x	|x|	Operación	Resultado
-3	3	3^3	27
2	2	2^3	8
-5	5	5^3	125
0	0	0^3	0

6. **Código en C**

```
#include <stdio.h>
#include <math.h>

int main() {
    float x, absoluto, resultado;

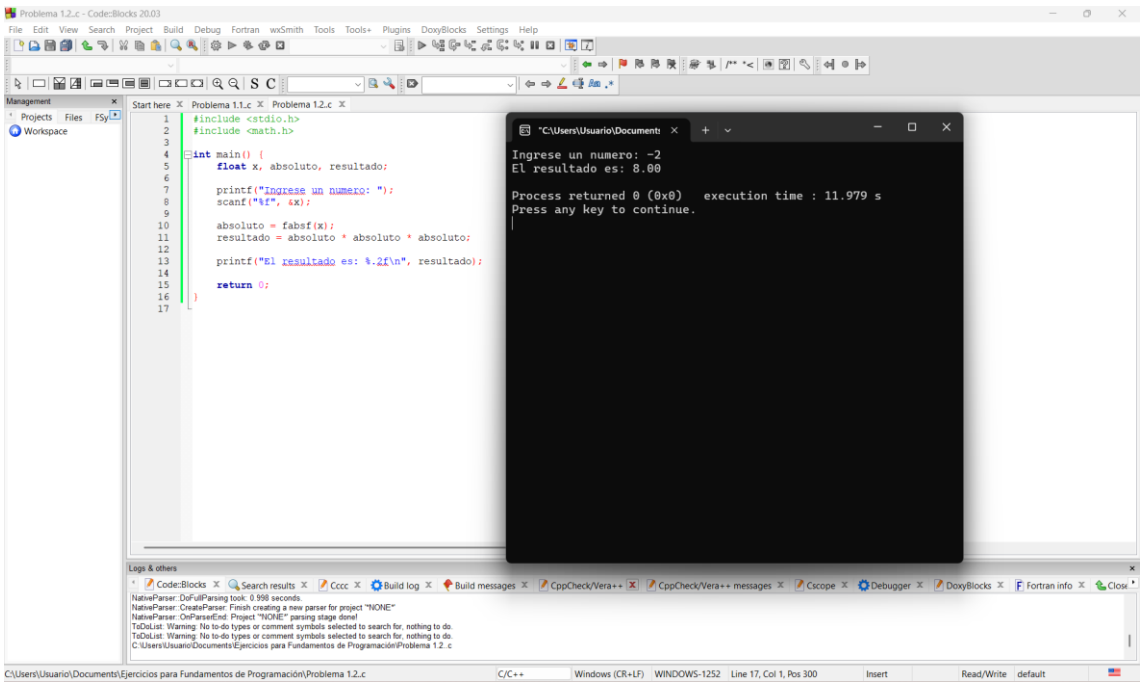
    printf("Ingrese un numero: ");
    scanf("%f", &x);

    absoluto = fabsf(x);
    resultado = absoluto * absoluto * absoluto;

    printf("El resultado es: %.2f\n", resultado);

    return 0;
}
```

7. Evidencia



Problema 1.3. Divisible

Leer 2 números enteros y determinar si el primero es divisible por el segundo.

1. **Análisis del problema**

Entrada: 2 números enteros a y b.

Proceso: Comprobar que b sea distinto de cero y calcular a MOD b.

Salida: Indicar si a es divisible por b.

2. **Tabla de objetos**

Objeto	Nombre	Valor	Tipo
Variable	a	Primer número	Entero
Variable	b	Segundo número	Entero
Variable	resto	a MOD b	Entero

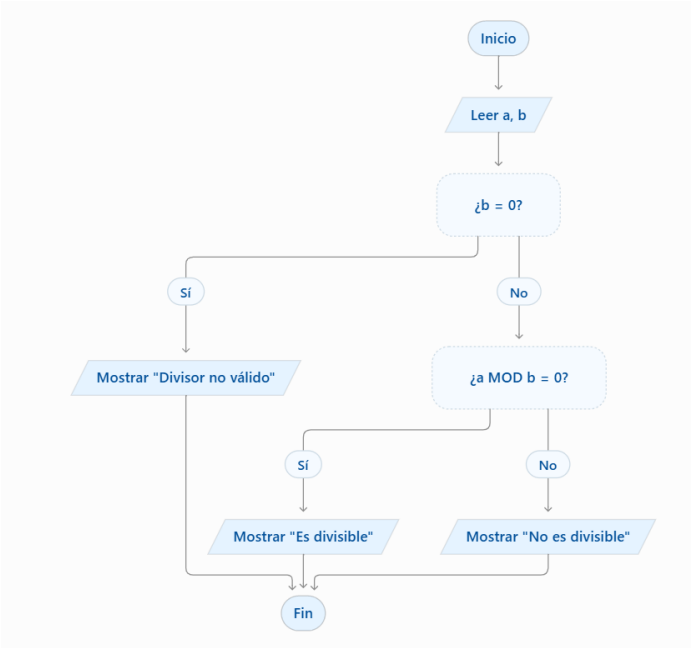
3. **Seudocódigo**

```
Inicio
  Definir a, b Como Entero

  Escribir "Ingrese el primer número:"
  Leer a
  Escribir "Ingrese el segundo número:"
  Leer b

  Si b = 0 Entonces
    Escribir "Error: el divisor no puede ser cero"
  SiNo
    Si a MOD b = 0 Entonces
      Escribir "Es divisible"
    SiNo
      Escribir "No es divisible"
    FinSi
  FinSi
FinAlgoritmo
```

4. **Diagrama de flujo**



5. Prueba de escritorio

a	b	Resto	Resultado
12	3	0	Divisible
15	4	3	No divisible
20	5	0	Divisible
8	0	No se calcula	Error

6. Código en C

```
#include <stdio.h>

int main() {
   int a, b;

   printf("Ingrese el primer numero: ");
   scanf("%d", &a);

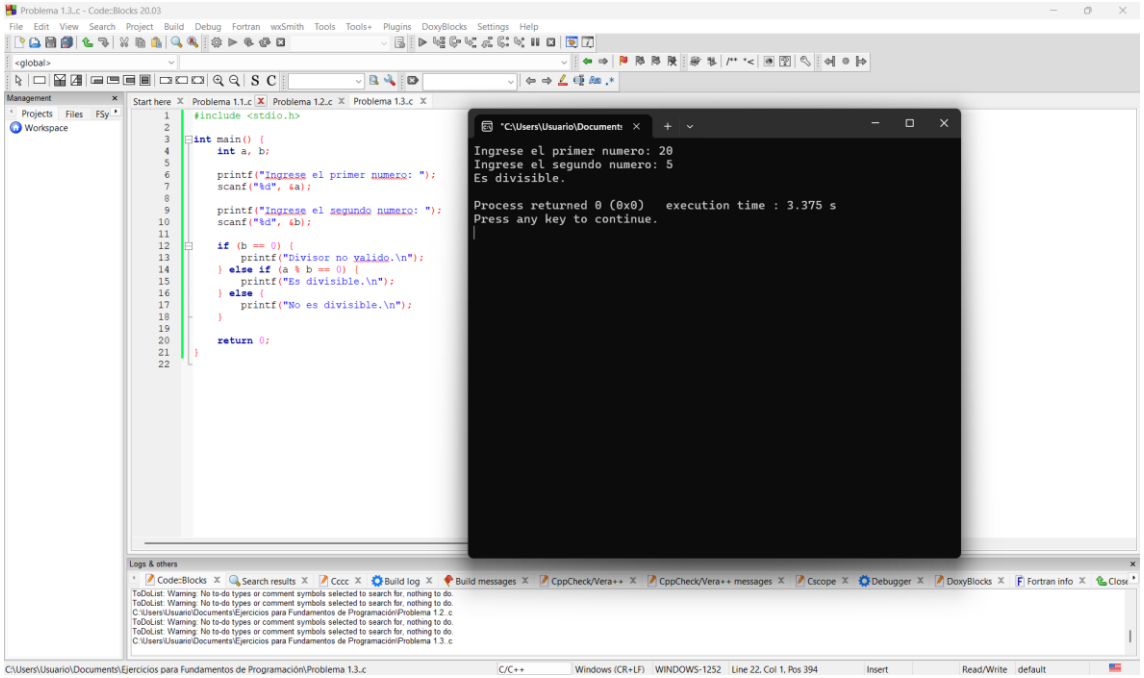
   printf("Ingrese el segundo numero: ");
   scanf("%d", &b);

   if (b == 0) {
      printf("Divisor no valido.\n");
   } else if (a % b == 0) {
```

```
        printf("Es divisible.\n");
    } else {
        printf("No es divisible.\n");
    }

    return 0;
}
```

7. Evidencia



Problema 1.4. Intervalo

Leer un número y determinar si pertenece al intervalo cerrado [0,10].

1. Análisis del problema

Entrada: Número real x.

Proceso: Comprobar simultáneamente x >=0 y x <=10.

Salida: Indicar si el numero pertenece al intervalo.

2. Tabla de objetos

Objeto	Nombre	Valor	Tipo
Variable	x	Ingresado por teclado	Real

3. Seudocódigo

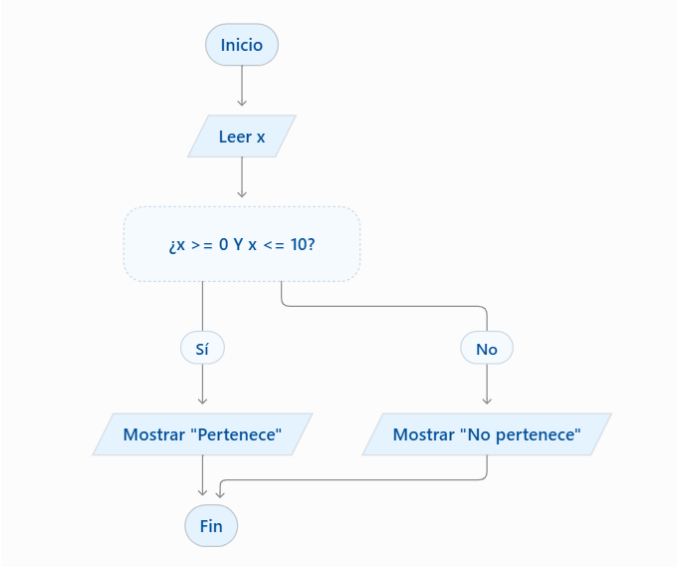
Inicio

```
  Definir x Como Real

  Escribir "Ingrese un número:"
  Leer x

  Si x >= 0 Y x <= 10 Entonces
    Escribir "Pertenece al intervalo"
  SiNo
    Escribir "No pertenece al intervalo"
  FinSi
FinAlgoritmo
```

4. Diagrama de flujo



5. Prueba de escritorio

Valor de X	¿X≤10?	Resultado
-2	Si	No pertenece
0	Si	Pertenece
5	Si	Pertenece
10	Si	Pertenece
12	Si	No pertenece

6. Código en C

```
#include <stdio.h>
```

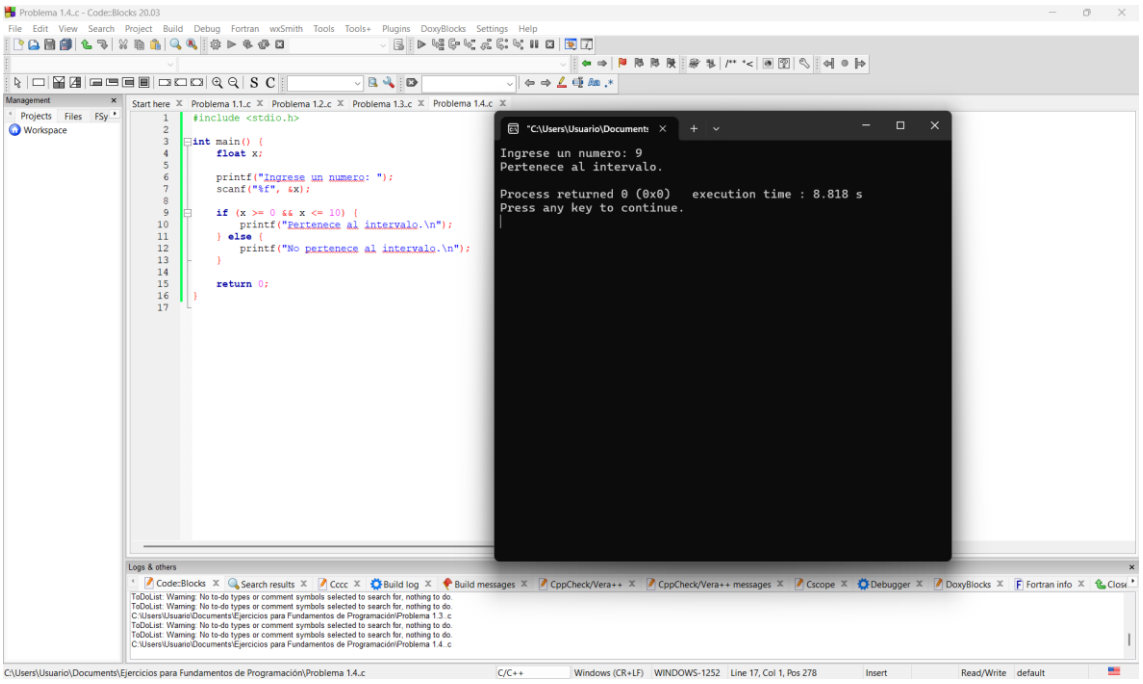
```
int main() {
    float x;

    printf("Ingrese un numero: ");
    scanf("%f", &x);

    if (x >= 0 && x <= 10) {
        printf("Pertenece al intervalo.\n");
    } else {
        printf("No pertenece al intervalo.\n");
    }

    return 0;
}
```

7. **Evidencia**



Problema 1.5. Conversión de unidades de tiempo

Desarrolla un programa que lea por teclado un valor entero C, interpretándolo como el número de segundos que dura un evento y calcule y muestre por pantalla cuántos días, horas, minutos y segundos representa.

1. **Análisis del problema**

Entrada: Un entero C con la cantidad de segundos (C >= 0).

Proceso: Descomponer C en enteros días, horas, minutos y segundos con división entera y módulo.

Salida: Los cuatro valores por pantalla.

2. Tabla de objetos

Objeto	Nombre	Valor
Segundos totales	C	Dato de entrada (>, 0)
Días completos	Días	c/ 86400
Segundos restantes	Resto	Se actualiza tras cada paso
Horas	Horas	resto/ 3600
Minutos	Minutos	resto/ 60
Segundos finales	Segundos	resto %60

3. Seudocódigo

```
Inicio
  Definir s, d, h, m, seg Como Entero

  Escribir "Ingrese la cantidad de segundos:"
  Leer s

  Si s < 0 Entonces
    Escribir "Valor inválido"
  SiNo
    d <- trunc(s / 86400)
    s <- s MOD 86400

    h <- trunc(s / 3600)
    s <- s MOD 3600

    m <- trunc(s / 60)
    seg <- s MOD 60

    Escribir "Días: ", d
    Escribir "Horas: ", h
```

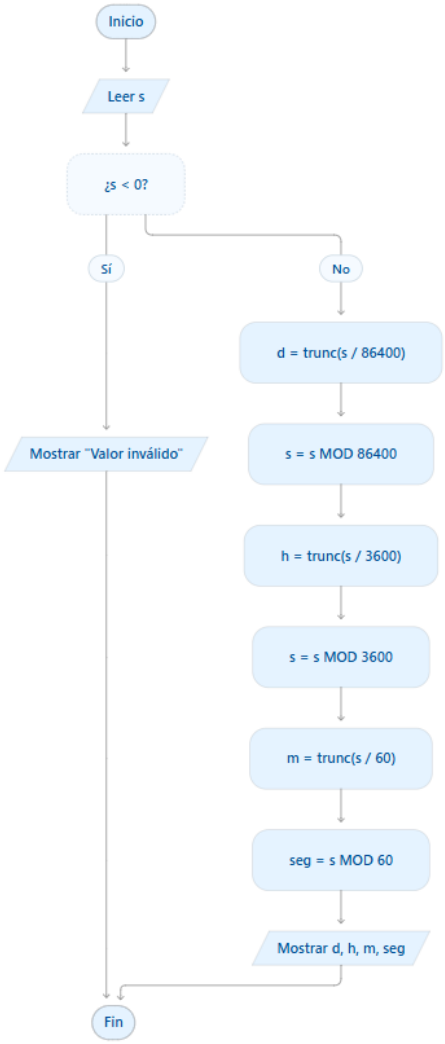
Escribir "Minutos: ", m

Escribir "Segundos: ", seg

FinSi

FinAlgoritmo

4. Diagrama de flujo



5. Prueba de escritorio

Paso	Operación	Resultado
1	Días = trunc(10000 / 86400)	0
2	Resto = 10000 MOD 86400	10000
3	Horas = trunc(10000 / 3600)	2
4	Resto = 10000 MOD 3600	2800
5	Minutos = trunc(2800 / 60)	46
6	Segundos = 2800 MOD 60	40

6. Código en C

```c
#include <stdio.h>

int main() {
   int s, dias, horas, minutos, segundos;

   printf("Ingrese la cantidad de segundos: ");
   scanf("%d", &s);

   if (s < 0) {
      printf("Valor invalido.\n");
   } else {
      dias = s / 86400;
      s = s % 86400;

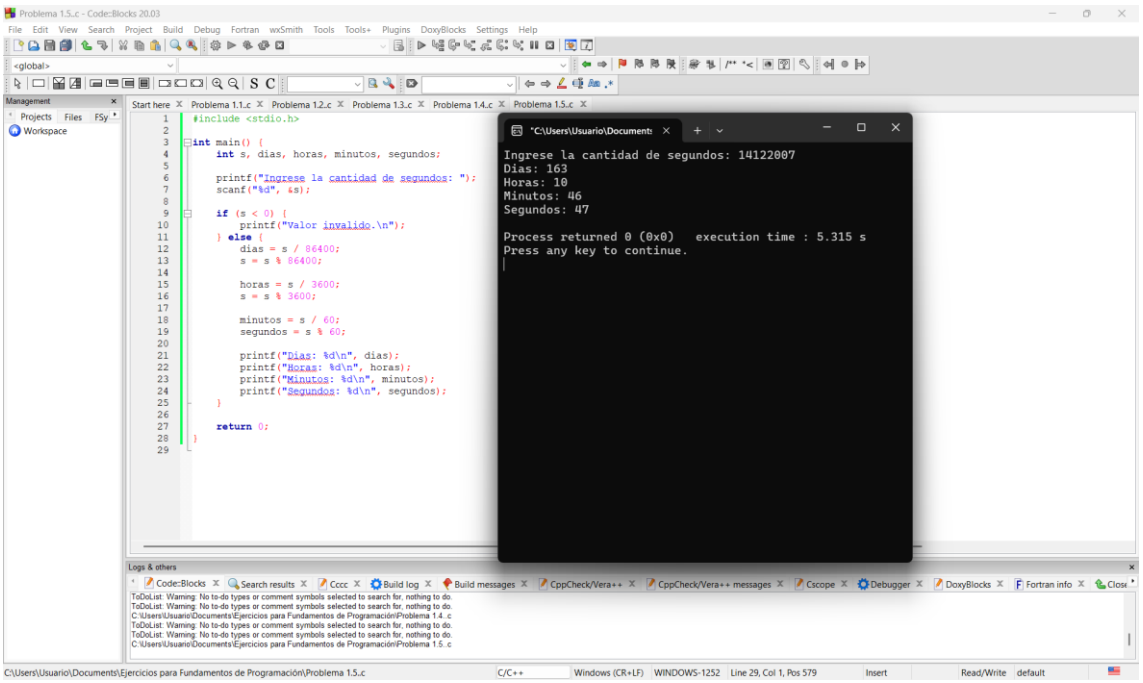
      horas = s / 3600;
      s = s % 3600;

      minutos = s / 60;
      segundos = s % 60;

      printf("Dias: %d\n", dias);
      printf("Horas: %d\n", horas);
      printf("Minutos: %d\n", minutos);
      printf("Segundos: %d\n", segundos);
   }

   return 0;
}
```

7. **Evidencia**



Problema 1.6. Máximo, mínimo y media de alturas

Desarrolla un programa que lea por teclado un número indeterminado de alturas, y deje de leer cuando se introduzca una negativa. El programa debe calcular e imprimir la media, el máx. y el mín. de las alturas introducidas.

1. **Análisis del problema**

Entrada: Una secuencia de alturas, la lectura termina al ingresar un número negativo.

Proceso: Acumular la suma y el contador para la media, actualizar el máx. y el mín.

Salida: Media, máximo y mínimo.

2. **Tabla de objetos**

Objeto	Nombre	Valor
Altura leída	h	Dato de entrada; negativo = fin
Suma de alturas	suma	Acumulador; inicia con la primera altura
Cantidad de alturas	cant	Contador; inicia en 1
Altura máx.	maxi	Se actualiza si h > max
Altura mín.	min	Se actualiza si h < min
Media	media	suma / cant

3. **Seudocódigo**

Inicio

Definir h, suma, mayor, menor, media Como Real

Definir cant Como Entero

```
suma <- 0
cant <- 0

Escribir "Ingrese una altura negativa para finalizar:"
Leer h

Mientras h >= 0 Hacer
   Si cant = 0 Entonces
      mayor <- h
      menor <- h
   SiNo
      Si h > mayor Entonces
         mayor <- h
      FinSi

      Si h < menor Entonces
         menor <- h
      FinSi
   FinSi

   suma <- suma + h
   cant <- cant + 1

   Leer h
FinMientras

Si cant > 0 Entonces
   media <- suma / cant

   Escribir "Media: ", media
   Escribir "Máximo: ", mayor
```

Escribir "Mínimo: ", menor

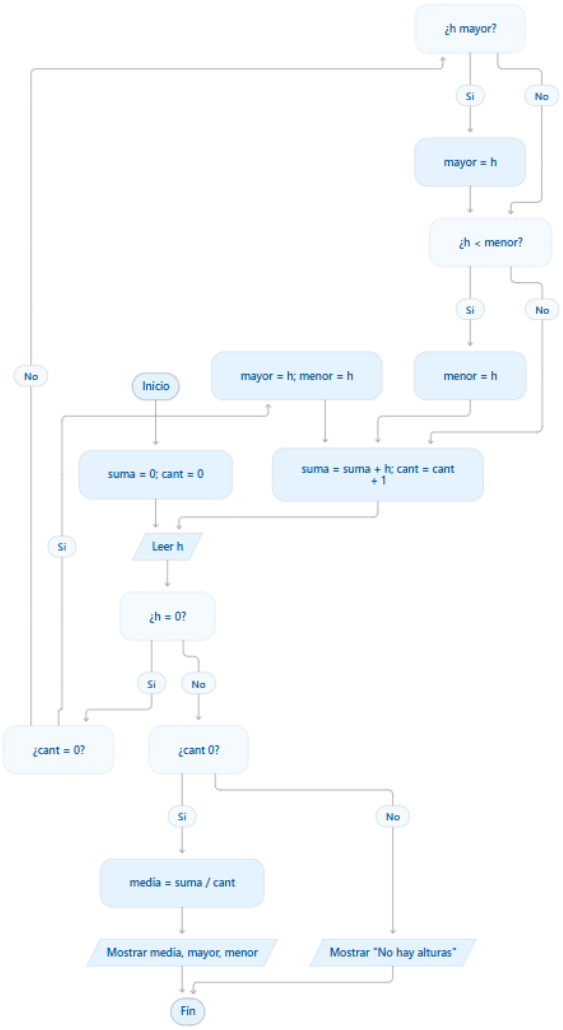
SiNo

Escribir "No se ingresaron alturas"

FinSi

FinAlgoritmo

4. Diagrama de flujo



5. Prueba de escritorio

Altura	Suma	Cantidad	Máximo	Mínimo
1,60	1,60	1	1,60	1,60
1,75	3,35	2	1,75	1,60
1,80	5,15	3	1,80	1,60
1,65	6,80	4	1,80	1,60
-1	6,80	4	1,80	1,60

6. Código en C

```c
#include <stdio.h>

int main() {
    float altura, suma = 0;
    float mayor = 0, menor = 0, promedio;
    int cantidad = 0;

    printf("Ingrese una altura (negativa para terminar): ");
    scanf("%f", &altura);

    while (altura >= 0) {
        if (cantidad == 0) {
            mayor = altura;
            menor = altura;
        } else {
            if (altura > mayor) {
                mayor = altura;
            }

            if (altura < menor) {
                menor = altura;
            }
        }

        suma = suma + altura;
        cantidad++;

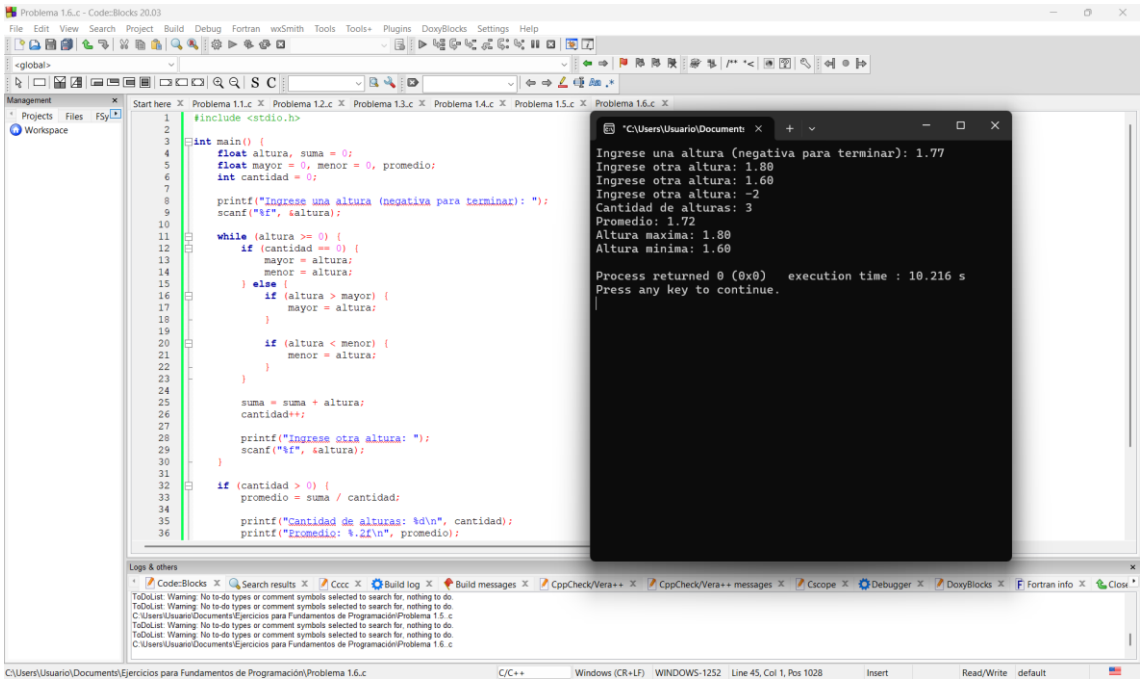
        printf("Ingrese otra altura: ");
        scanf("%f", &altura);
    }
```

```
    if (cantidad > 0) {
        promedio = suma / cantidad;


        printf("Cantidad de alturas: %d\n", cantidad);
        printf("Promedio: %.2f\n", promedio);
        printf("Altura maxima: %.2f\n", mayor);
        printf("Altura minima: %.2f\n", menor);
    } else {
        printf("No se ingresaron alturas.\n");
    }


    return 0;
}
```

7. Evidencia



Problema 1.7. Raíz cuadrada entera

Desarrolla un programa que calcule la raíz cuadrada entera por defecto de un número N positivo dado, que se define como el mayor número entero $R > 0$ tal que $R^2 \leq N$.

1. Análisis del problema

Entrada: Un entero positivo N.

Proceso: Partiendo de i = 1, incrementar i mientras $i^2 \leq N$. Al salir, i es el primero.

Salida: R, el mayor entero con $R^2 \leq N$.

2. Tabla de objetos

Objeto	Nombre	Valor	Tipo
Número dado	N	Dato de entrada (> 0)	Entero
Contador de prueba	i	Inicia en 1, aumenta de 1 en 1	Entero
Raíz entera por defecto	R	i - 1	Entero

3. Seudocódigo

```
Inicio
  Definir n, i, r Como Entero

  Escribir "Ingrese un entero positivo:"
  Leer n

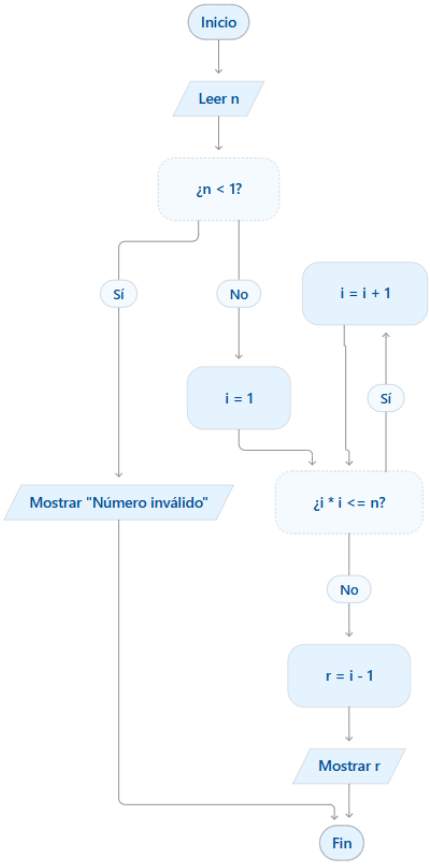
  Si n < 1 Entonces
    Escribir "Error: el número debe ser positivo"
  SiNo
    i <- 1

    Mientras i * i <= n Hacer
      i <- i + 1
    FinMientras

    r <- i - 1

    Escribir "Raíz cuadrada entera: ", r
  FinSi
FinAlgoritmo
```

4. Diagrama de flujo



5. Prueba de escritorio

N	Comprobación	Resultado
1	1^2≤1	1
15	3^2≤15, pero 4^2>15	3
16	4^2≤16, pero 5^2>16	4
20	4^2≤20, pero 5^2>20	4
0	No es positivo	Error

6. Código en C

```
#include <stdio.h>


int main() {
   int n, i = 1, resultado;


   printf("Ingrese un numero entero no negativo: ");
   scanf("%d", &n);


   if (n < 0) {
```

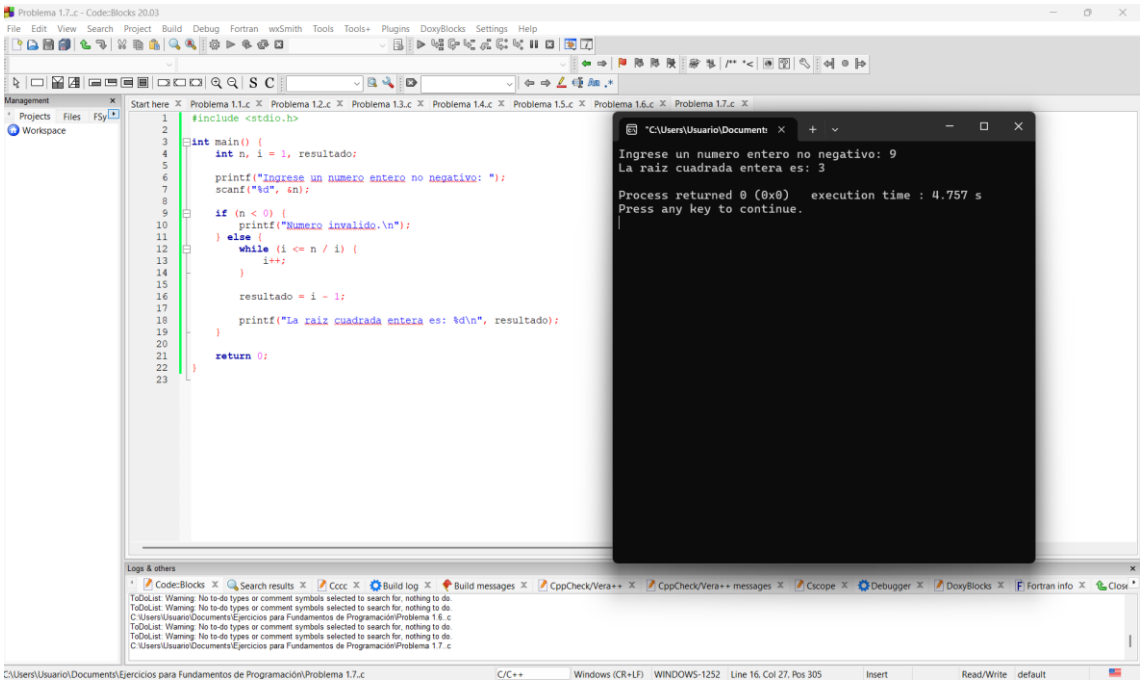
```
        printf("Numero invalido.\n");
    } else {
        while (i <= n / i) {
            i++;
        }

        resultado = i - 1;

        printf("La raiz cuadrada entera es: %d\n", resultado);
    }

    return 0;
}
```

7. Evidencia



Problema 1.8. Número primo

1. Análisis del problema

Entrada: Un entero N>1.

Proceso: Un número es primo si no tiene divisores entre los que se prueban. Se prueban los divisores d=2,3, … mientras d x d ≤ N. Si alguno cumple N mod d=0, no es primo.

Salida: Mensaje «es primo» o «no es primo».

2. Tabla de objetos

Objeto	Nombre	Valor	Tipo
Número a evaluar	n	Dato de entrada	Entero
Divisor de prueba	d	Inicia en 2	Entero
Bandera de primalidad	eprimo	Verdadero al inicio; falso si se encuentra divisor	Entero

3. Seudocódigo

```
Inicio
  Definir n, d Como Entero
  Definir primo Como Logico

  Escribir "Ingrese un entero mayor que 1:"
  Leer n

  Si n <= 1 Entonces
    Escribir "El número debe ser mayor que 1"
  SiNo
    primo <- Verdadero
    d <- 2

    Mientras d * d <= n Y primo Hacer
      Si n MOD d = 0 Entonces
        primo <- Falso
      FinSi

      d <- d + 1
    FinMientras

    Si primo Entonces
      Escribir n, " es primo"
    SiNo
```

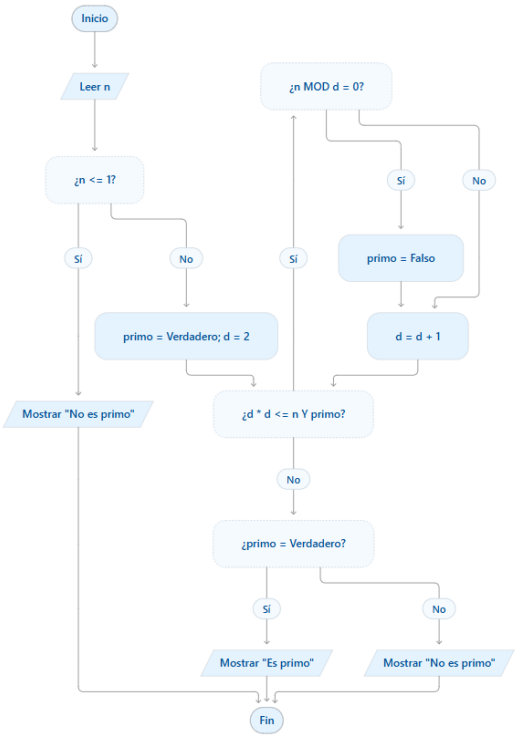
Escribir n, " no es primo"

FinSi

FinSi

FinAlgoritmo

4. Diagrama de flujo



5. Prueba de escritorio

Número	Divisores de prueba	Resultado
2	No necesita probar divisores	Primo
7	2	Primo
9	2, 3	No es primo
15	2, 3	No es primo
17	2, 3, 4	Primo
1	No valido	No es primo

6. Código en C

```
#include <stdio.h>


int main() {
   int n, divisor = 2, primo = 1;
```

```
    printf("Ingrese un numero entero: ");
    scanf("%d", &n);

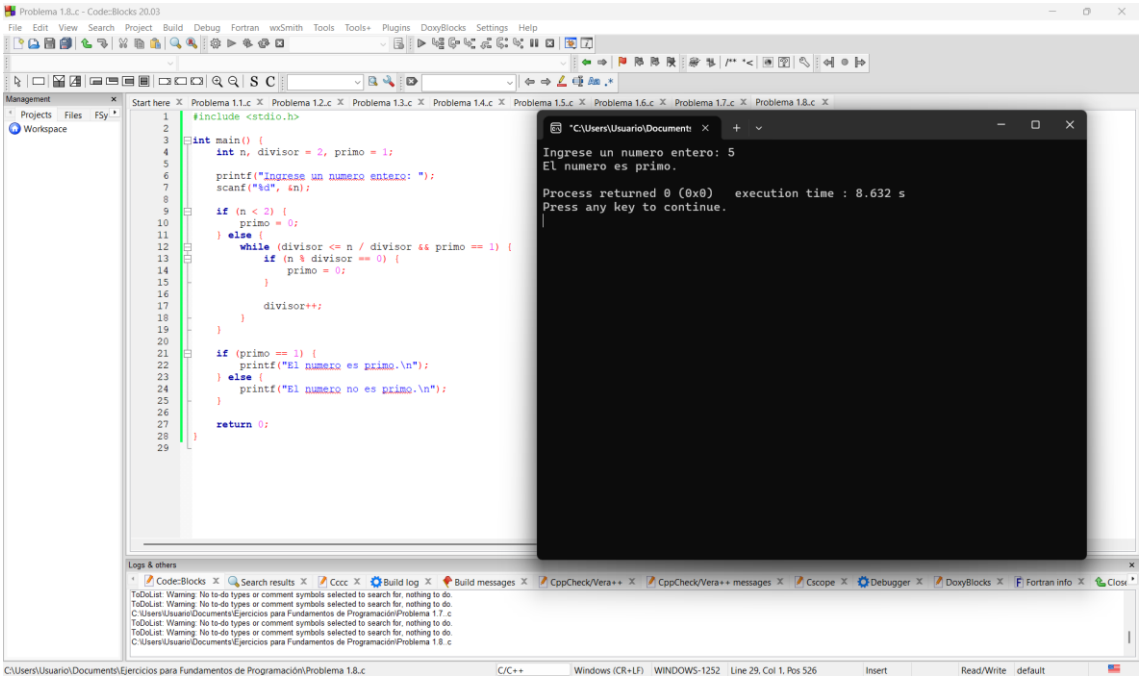
    if (n < 2) {
        primo = 0;
    } else {
        while (divisor <= n / divisor && primo == 1) {
            if (n % divisor == 0) {
                primo = 0;
            }

            divisor++;
        }
    }

    if (primo == 1) {
        printf("El numero es primo.\n");
    } else {
        printf("El numero no es primo.\n");
    }

    return 0;
}
```

7. **Evidencia**



Problema 1.9. Números primos comprendidos entre dos números dados

Desarrolla un programa que muestre por pantalla todos los números primos comprendidos entre dos números enteros introducidos por teclado.

1. **Análisis del problema**

Entrada: Dos enteros a y b.

Proceso: Recorrer cada número n desde a hasta b y aplicar a cada uno la prueba de primalidad del problema 1.8.

Salida: Todos los primos del rango. Si no hay ninguno, se informa.

2. **Tabla de objetos**

Objeto	Nombre	Valor	Tipo
Primer extremo	a	Dato de entrada	Entero
Segundo extremo	b	Dato de entrada	Entero
Auxiliar de intercambio	aux	Guarda temporalmente un valor	Entero
Número de revisión	n	Va de a hasta b	Entero
Divisor de prueba	d	Inicia en 2 para cada n	Entero
Bandera de primalidad	esprimo	Verdadero o Falso	Lógico
Cantidad de primos hallados	hallado	Contador, inicia en 0	Entero

3. **Seudocódigo**

Inicio

```
Definir a, b, n, d Como Entero
Definir primo Como Logico
Definir aux Como Entero
Definir cantidad Como Entero

Escribir "Ingrese el primer número:"
Leer a
Escribir "Ingrese el segundo número:"
Leer b

Si a > b Entonces
   aux <- a
   a <- b
   b <- aux
FinSi

cantidad <- 0

Para n <- a Hasta b Hacer
   Si n > 1 Entonces
      primo <- Verdadero
      d <- 2

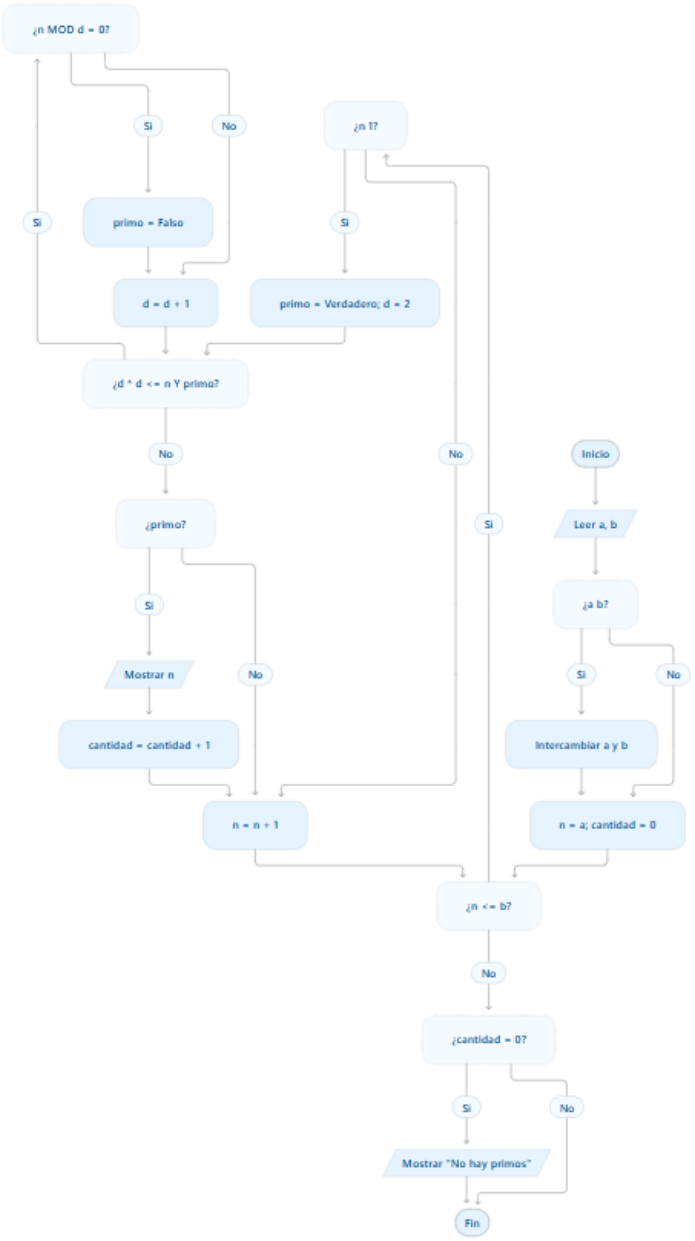
      Mientras d * d <= n Y primo Hacer
         Si n MOD d = 0 Entonces
            primo <- Falso
         FinSi

         d <- d + 1
      FinMientras
```

```
            Si primo Entonces
                Escribir n
                cantidad <- cantidad + 1
            FinSi
        FinSi
    FinPara

    Si cantidad = 0 Entonces
        Escribir "No hay números primos en el intervalo"
    FinSi
FinAlgoritmo
```

4. **Diagrama de flujo**



5. Prueba de escritorio

Extremo Inicial	Extremo final	Números primos encontrados
1	10	2, 3, 5, 7
10	20	11, 13, 17, 19
14	16	Ninguno
20	10	11, 13, 17, 19
-3	3	2, 3

6. Código en C

```
#include <stdio.h>
```

```
int main() {
    int a, b, aux;
    int n, divisor, primo;

    printf("Ingrese el primer numero: ");
    scanf("%d", &a);

    printf("Ingrese el segundo numero: ");
    scanf("%d", &b);

    if (a > b) {
        aux = a;
        a = b;
        b = aux;
    }

    printf("Numeros primos entre %d y %d:\n", a, b);

    for (n = a; ; n++) {
        primo = 1;

        if (n < 2) {
            primo = 0;
        } else {
            divisor = 2;

            while (divisor <= n / divisor && primo == 1) {
                if (n % divisor == 0) {
                    primo = 0;
                }
```

```
            divisor++;
        }
    }

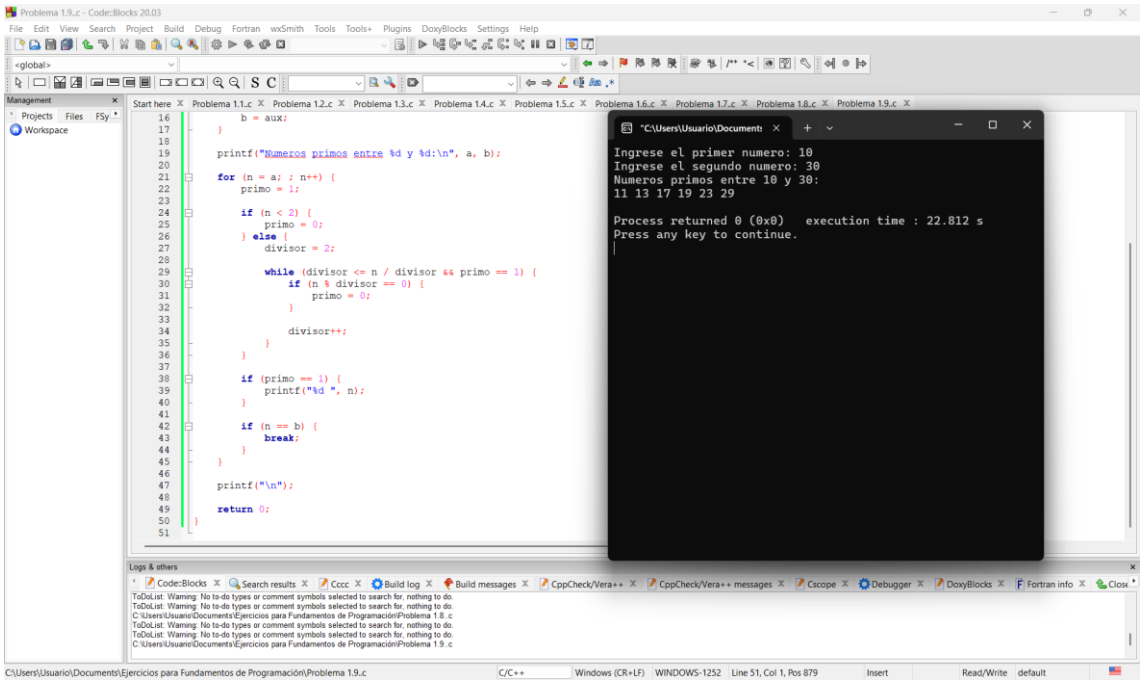
    if (primo == 1) {
        printf("%d ", n);
    }

    if (n == b) {
        break;
    }
  }

  printf("\n");

  return 0;
}
```

7. Evidencia



Problema 1.10. Factorial

Desarrolla un programa que lea un número n ≥ 0 y calcule n! = 1 x 2 x … x n, sabiendo que 0! = 1.

1. **Análisis del problema**

Entrada: Un entero n ≥ 0.

Proceso: n! = 1 x 2 x … x n, con 0! = 1. Se usa un acumulador que inicia en 1 y se multiplica por 1 en cada vuelta, con i ≤ n.

Salida: El valor de n!.

2. **Tabla de objetos**

Objeto	Nombre	Valor	Tipo
Número dado	n	Dato de entrada (≥ 0)	Entero
Contador del ciclo	i	De 1 hasta n	Entero
Acumulador del factorial	fact	Inicio en 1; fact = fact x i	Entero largo

3. **Seudocódigo**

```
Inicio
  Definir n, i Como Entero
  Definir fact Como Entero

  Escribir "Ingrese un entero no negativo:"
  Leer n

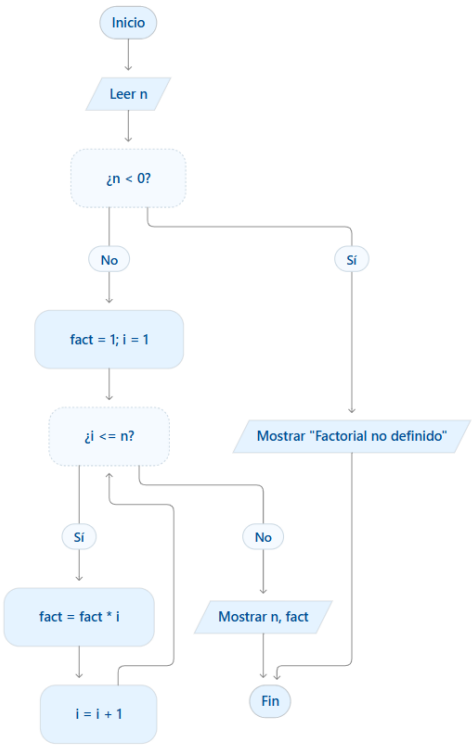
  Si n < 0 Entonces
    Escribir "Error: el factorial no existe para enteros negativos"
  SiNo
    fact <- 1

    Para i <- 1 Hasta n Hacer
      fact <- fact * i
    FinPara

    Escribir n, "! = ", fact
  FinSi
```

FinAlgoritmo

4. Diagrama de flujo



5. Prueba de escritorio

Caso	n	Calculo	Resultado
Límite inferior	0	0!=1	1
Primer factorial	1	1!=1	1
Número pequeño	5	5!=1 x 2 x 3 x 4 x 5	120
Número mediano	10	10!	3 628 800
Número grande	20	20!	2 432 902 008 176 640 000
Entrada invalida	-3	No se calcula	Error

6. Código en C

```
#include <stdio.h>


int main() {
    int n, i;
    unsigned long long factorial = 1;

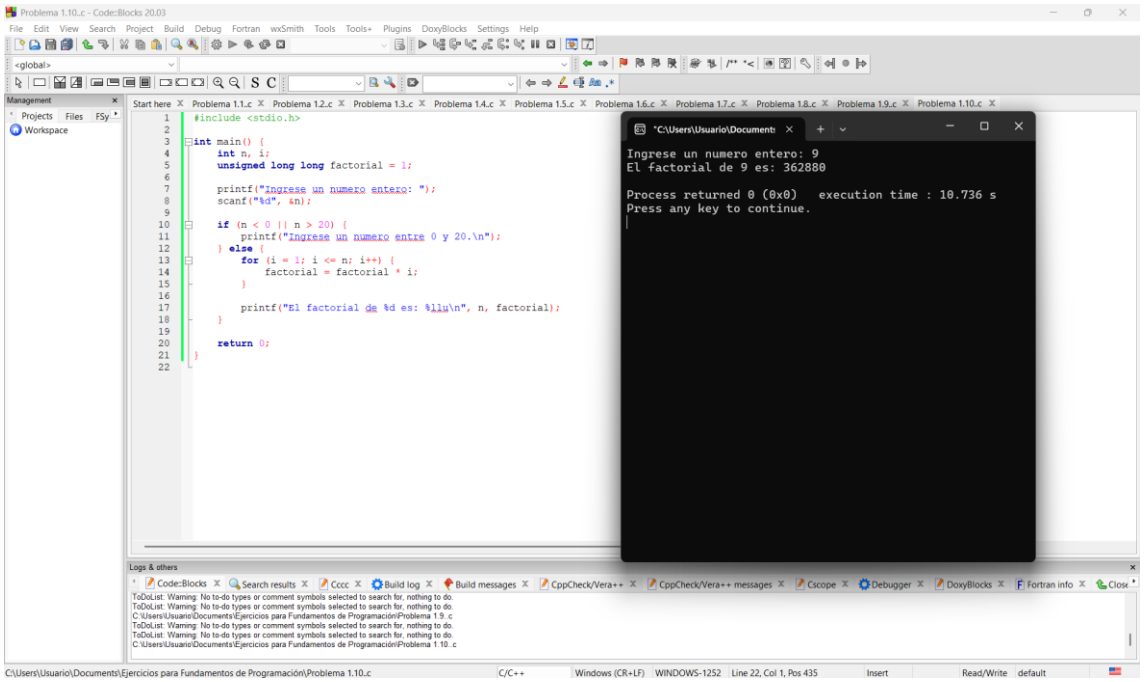

    printf("Ingrese un numero entero: ");
    scanf("%d", &n);
```

```
    if (n < 0 || n > 20) {
        printf("Ingrese un numero entre 0 y 20.\n");
    } else {
        for (i = 1; i <= n; i++) {
            factorial = factorial * i;
        }

        printf("El factorial de %d es: %llu\n", n, factorial);
    }

    return 0;
}
```

7. Evidencia



Problema 1.11. Serie de Fibonacci

Desarrolla un programa que lea un entero n ≥ 1 y muestre los primeros n términos de la serie de Fibonacci (0, 1, 1, 2, 3, 5, 8, …), donde cada término es la suma de los dos anteriores.

1. Análisis del problema

Entrada: un entero n ≥ 1.

Proceso: generar los primeros n términos de Fibonacci. Cada término se obtiene sumando los dos anteriores.

Salida: mostrar los primeros n términos de la serie.

2. **Tabla de objetos**

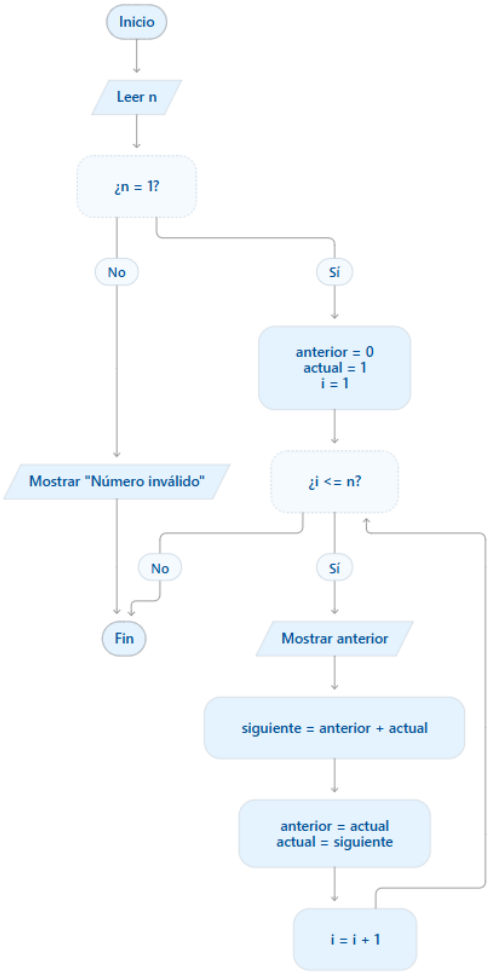
Objeto	Tipo de dato	Descripción
n	Entero	Cantidad de términos que se mostrarán.
anterior	Entero	Término anterior de la serie.
actual	Entero	Término actual de la serie.
siguiente	Entero	Suma de los dos términos anteriores.
i	Entero	Contador de términos.

3. **Seudocódigo**

```
Inicio
   Leer n
   anterior <- 0
   actual <- 1

   Para i <- 1 Hasta n Hacer
      Escribir anterior
      siguiente <- anterior + actual
      anterior <- actual
      actual <- siguiente
   FinPara
Fin
```

4. **Diagrama de flujo**



5. Prueba de escritorio

Iteración	anterior	siguiente	Nuevo (anterior)	Nuevo (actual)
1	0	1	1	1
2	1	2	1	2
3	1	3	2	3
4	2	5	3	5
5	3	8	5	8
6	5	13	8	13
7	8	21	13	21

6. Código en C

```
#include <stdio.h>


int main() {
   int n, anterior = 0, actual = 1;
   int siguiente, i;
```

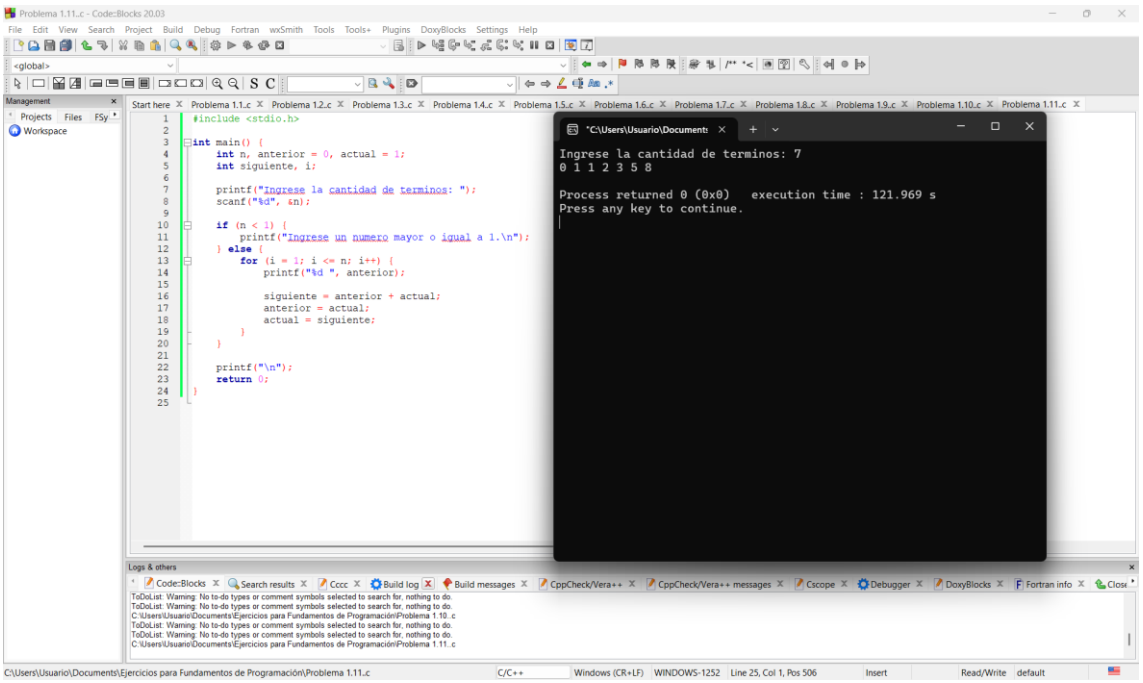
```
    printf("Ingrese la cantidad de terminos: ");
    scanf("%d", &n);

    if (n < 1) {
        printf("Ingrese un numero mayor o igual a 1.\n");
    } else {
        for (i = 1; i <= n; i++) {
            printf("%d ", anterior);

            siguiente = anterior + actual;
            anterior = actual;
            actual = siguiente;
        }
    }

    printf("\n");
    return 0;
}
```

7. Evidencia



Problema 1.12. Suma de dígitos

Desarrolla un programa que lea un entero positivo y calcule la suma de sus dígitos.

1. **Análisis del problema**

Entrada: un entero positivo.

Proceso: obtener el último dígito con el operador %10, sumarlo y eliminarlo mediante la división entera entre 10. Repetir hasta que el número sea cero.

Salida: mostrar la suma de los dígitos.

2. **Tabla de objetos**

Objeto	Tipo de dato	Descripción
n	Entero	Número ingresado por el usuario.
digito	Entero	Último dígito del número.
suma	Entero	Acumulador de los dígitos.

3. **Seudocódigo**

Inicio

Leer n

suma <- 0

Mientras n > 0 Hacer

digito <- n MOD 10

suma <- suma + digito

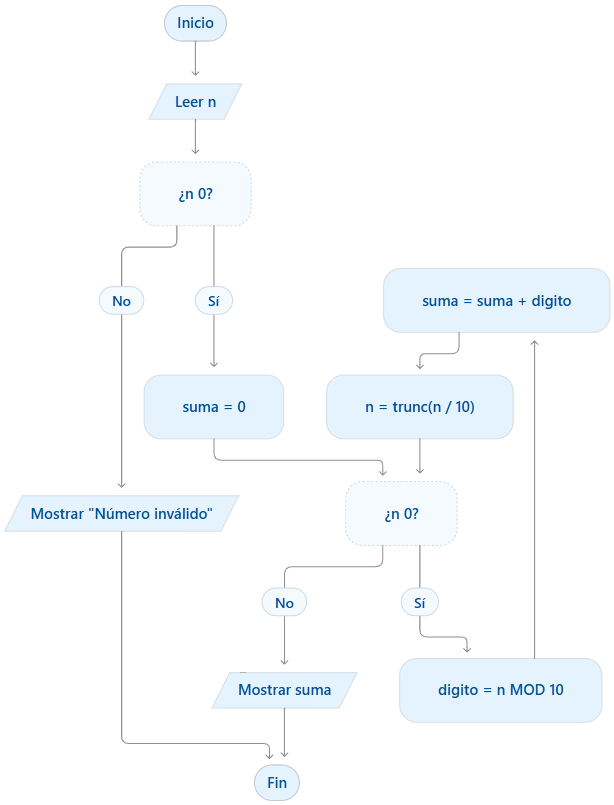
n <- trunc(n / 10)

FinMientras

Escribir suma

Fin

4. Diagrama de flujo



5. Prueba de escritorio

Iteración	n	digito = n % 10	Suma acumulada	n = n / 10
1	4825	5	5	482
2	482	2	7	48
3	48	8	15	4
4	4	4	19	0

6. Código en C

```
#include <stdio.h>

int main() {
   int n, digito, suma = 0;
```

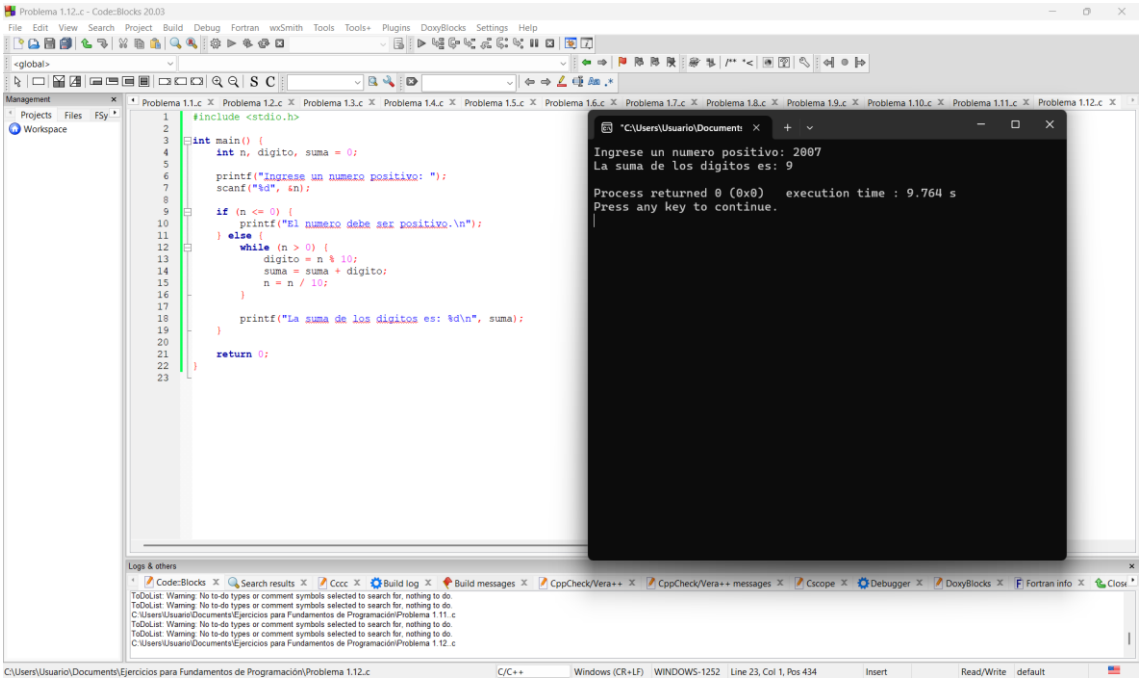
```
    printf("Ingrese un numero positivo: ");
    scanf("%d", &n);

    if (n <= 0) {
        printf("El numero debe ser positivo.\n");
    } else {
        while (n > 0) {
            digito = n % 10;
            suma = suma + digito;
            n = n / 10;
        }

        printf("La suma de los digitos es: %d\n", suma);
    }

    return 0;
}
```

7. Evidencia



Problema 1.13. Número capicúa

Desarrolla un programa que lea un entero positivo y determine si es capicúa, es decir, si se lee igual de izquierda a derecha que de derecha a izquierda.

1. **Análisis del problema**

Entrada: un entero positivo.

Proceso: guardar el número original, construir su inverso extrayendo los dígitos y comparar ambos números.

Salida: indicar si el número es capicúa o no.

2. **Tabla de objetos**

Objeto	Tipo de dato	Descripción
n	Entero	Número que se analizará.
original	Entero	Copia del número original.
invertido	Entero	Número formado en orden inverso.
digito	Entero	Último dígito extraído.

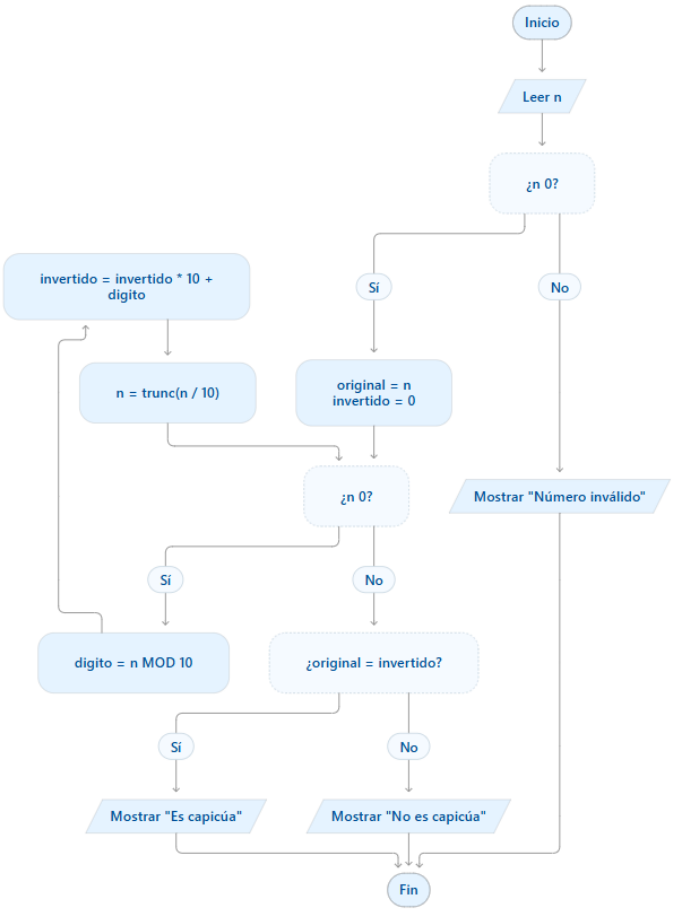
3. **Seudocódigo**

```
Inicio
  Leer n
  original <- n
  invertido <- 0

  Mientras n > 0 Hacer
    digito <- n MOD 10
    invertido <- invertido * 10 + digito
    n <- trunc(n / 10)
  FinMientras

  Si original = invertido Entonces
    Escribir "Es capicua"
  SiNo
    Escribir "No es capicua"
  FinSi
Fin
```

4. Diagrama de flujo



5. Prueba de escritorio

Iteración	n (inicial)	digito	invertido	n (final)
1	12321	1	1	1232
2	1232	2	12	123
3	123	3	123	12
4	12	2	1232	1
5	1	1	12321	0

6. Código en C

```
#include <stdio.h>


int main() {
   int n, original, invertido = 0, digito;


   printf("Ingrese un numero positivo: ");
   scanf("%d", &n);
```

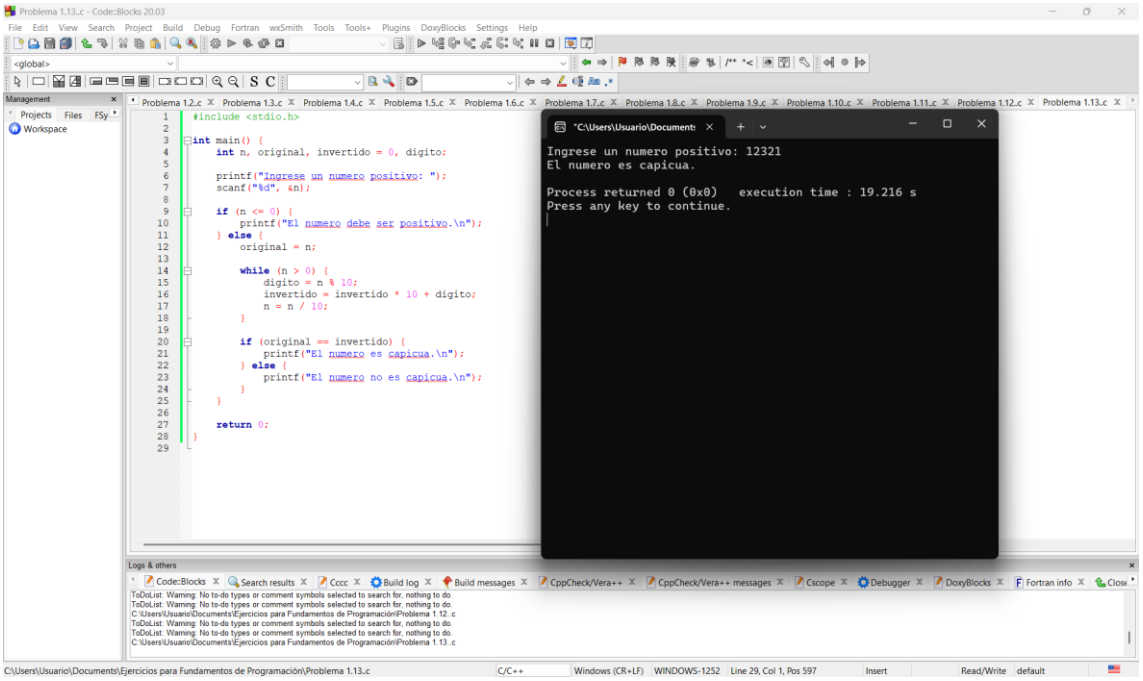
```
    if (n <= 0) {
        printf("El numero debe ser positivo.\n");
    } else {
        original = n;

        while (n > 0) {
            digito = n % 10;
            invertido = invertido * 10 + digito;
            n = n / 10;
        }

        if (original == invertido) {
            printf("El numero es capicua.\n");
        } else {
            printf("El numero no es capicua.\n");
        }
    }

    return 0;
}
```

7. Evidencia



Problema 1.14. Máximo común divisor (algoritmo de Euclides)

Desarrolla un programa que lea dos enteros positivos a y b y calcule su máximo común divisor con el algoritmo de Euclides: mientras b =/ 0, reemplaza (a,b) por (b, a % b). Al terminar, a es el MCD.

1. **Análisis del problema**

Entrada: dos enteros positivos, a y b.

Proceso: mientras b sea diferente de cero, calcular el resto de a dividido entre b; luego asignar b a a y el resto a b.

Salida: mostrar el máximo común divisor (MCD).

2. **Tabla de objetos**

Objeto	Tipo de dato	Descripción
a	Entero	Primer número. Al final contiene el MCD.
b	Entero	Segundo número y divisor en el algoritmo.
resto	Entero	Resto de dividir a entre b.

3. **Seudocódigo**

Inicio

Leer a, b

Mientras b <> 0 Hacer

resto <- a MOD b

a <- b

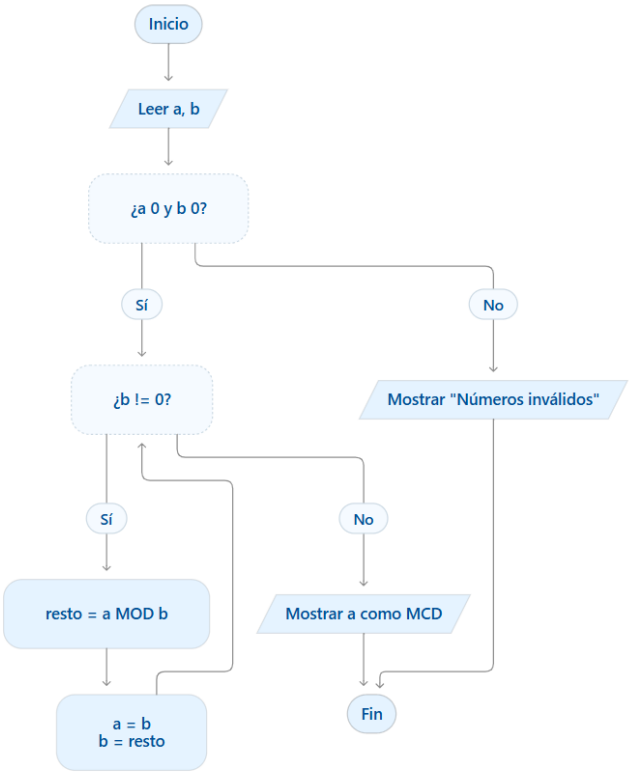
b <- resto

FinMientras

Escribir a

Fin

4. Diagrama de flujo



5. Prueba de escritorio

Iteración	a (inicial)	b (inicial)	resto = a % b	Nuevo a	Nuevo b
1	48	18	12	18	12
2	18	12	6	12	6
3	12	6	0	6	0

6. Código en C

```
#include <stdio.h>

int main() {
    int a, b, resto;
```

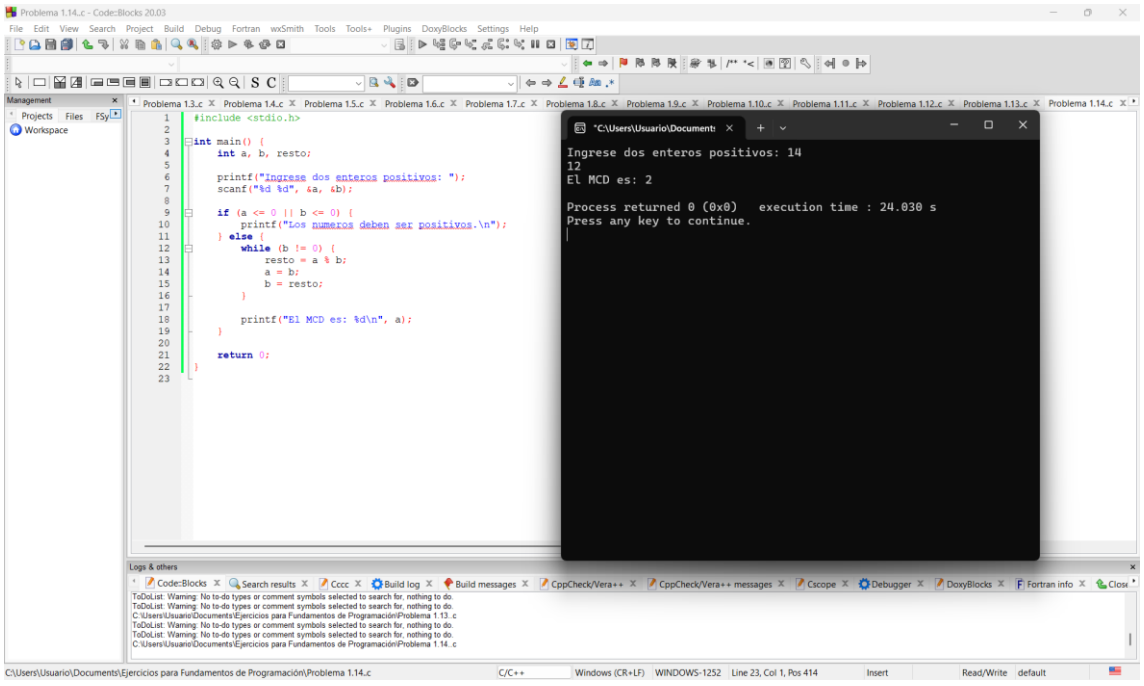
```
    printf("Ingrese dos enteros positivos: ");
    scanf("%d %d", &a, &b);

    if (a <= 0 || b <= 0) {
        printf("Los numeros deben ser positivos.\n");
    } else {
        while (b != 0) {
            resto = a % b;
            a = b;
            b = resto;
        }

        printf("El MCD es: %d\n", a);
    }

    return 0;
}
```

7. Evidencia



Problema 1.15. Año bisiesto

Desarrolla un programa que lea un año y determine si es bisiesto. Un año es bisiesto si es divisible entre 4, excepto los años divisibles entre 100, que solo lo son si también son divisibles entre 400.

1. **Análisis del problema**

Entrada: un año.

Proceso: verificar si es divisible entre 400 o si es divisible entre 4, pero no entre 100.

Salida: indicar si el año es bisiesto o no.

2. **Tabla de objetos**

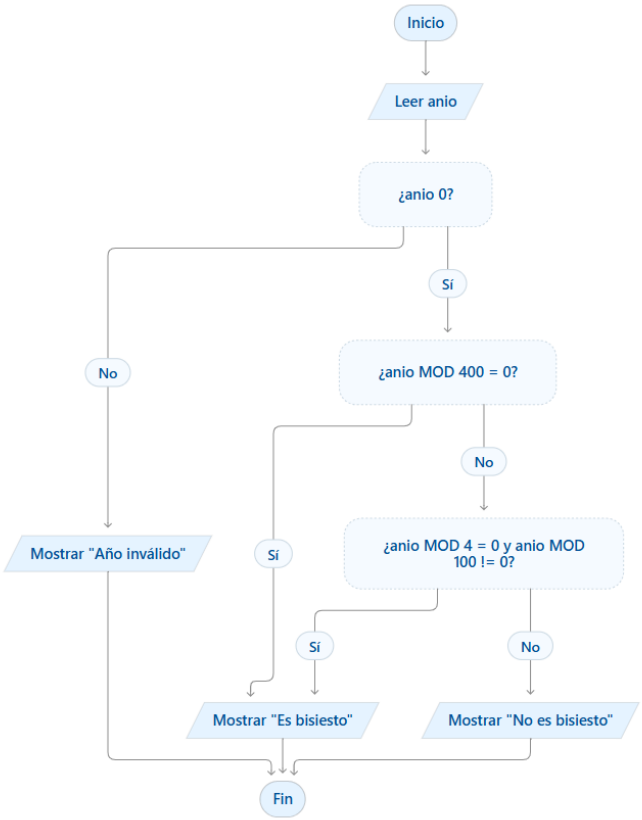
Objeto	Tipo de dato	Descripción
anio	Entero	Año ingresado por el usuario.

3. **Seudocódigo**

```
Inicio
  Leer anio

  Si anio MOD 400 = 0 O
    (anio MOD 4 = 0 Y anio MOD 100 <> 0) Entonces
     Escribir "Es bisiesto"
  SiNo
     Escribir "No es bisiesto"
  FinSi
Fin
```

4. **Diagrama de flujo**



5. Prueba de escritorio

Año	¿Divisible entre 400?	¿Divisible entre 4 y no entre 100?	Resultado
2024	No	Si	Es bisiesto
1900	No	No	No es bisiesto
2000	Si	No es necesario comprobar la segunda condición	Es bisiesto

6. Código en C

```
#include <stdio.h>

int main() {
    int anio;

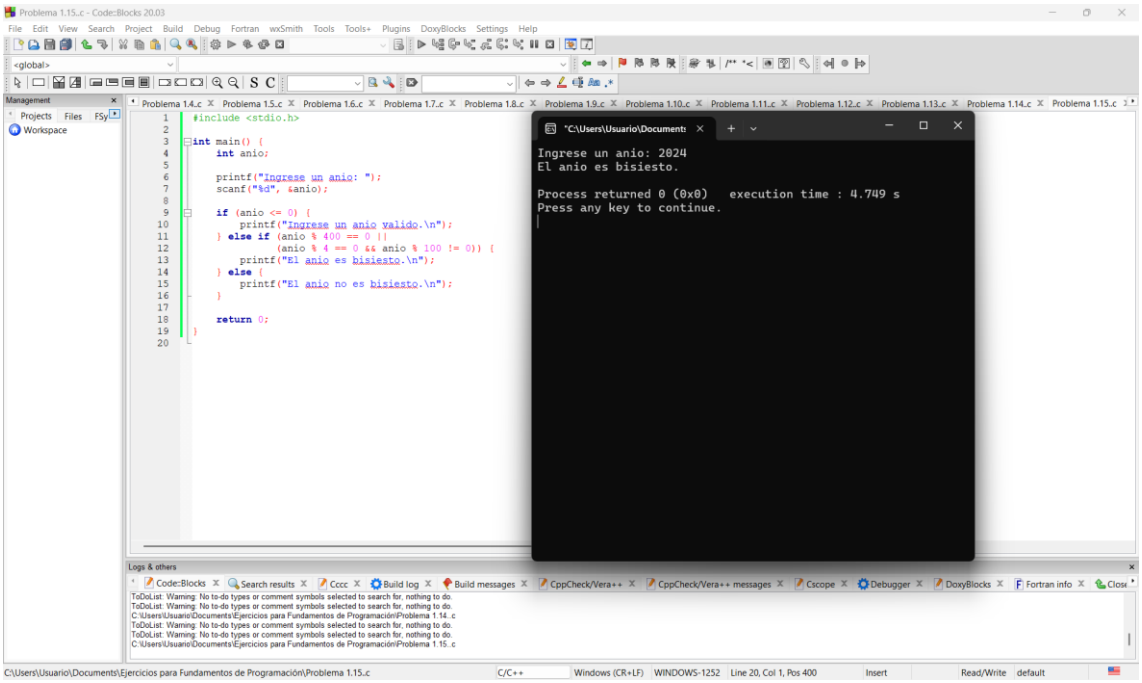
    printf("Ingrese un anio: ");
    scanf("%d", &anio);

    if (anio <= 0) {
```

```
        printf("Ingrese un anio valido.\n");
    } else if (anio % 400 == 0 ||
            (anio % 4 == 0 && anio % 100 != 0)) {
        printf("El anio es bisiesto.\n");
    } else {
        printf("El anio no es bisiesto.\n");
    }

    return 0;
}
```

7. Evidencia



Problema 1.16. Número perfecto

Desarrolla un programa que lea un entero positivo y determine si es perfecto, es decir, si es igual a la suma de sus divisores propios (los menores que él).

1. Análisis del problema

Entrada: un entero positivo.

Proceso: recorrer los números desde 1 hasta n - 1, sumar los que sean divisores exactos de n y comparar la suma con el número original.

Salida: indicar si el número es perfecto.

2. Tabla de objetos

Objeto	Tipo de dato	Descripción
n	Entero	Número que se analizará.
d	Entero	Posible divisor de n.
suma	Entero	Suma de los divisores propios.

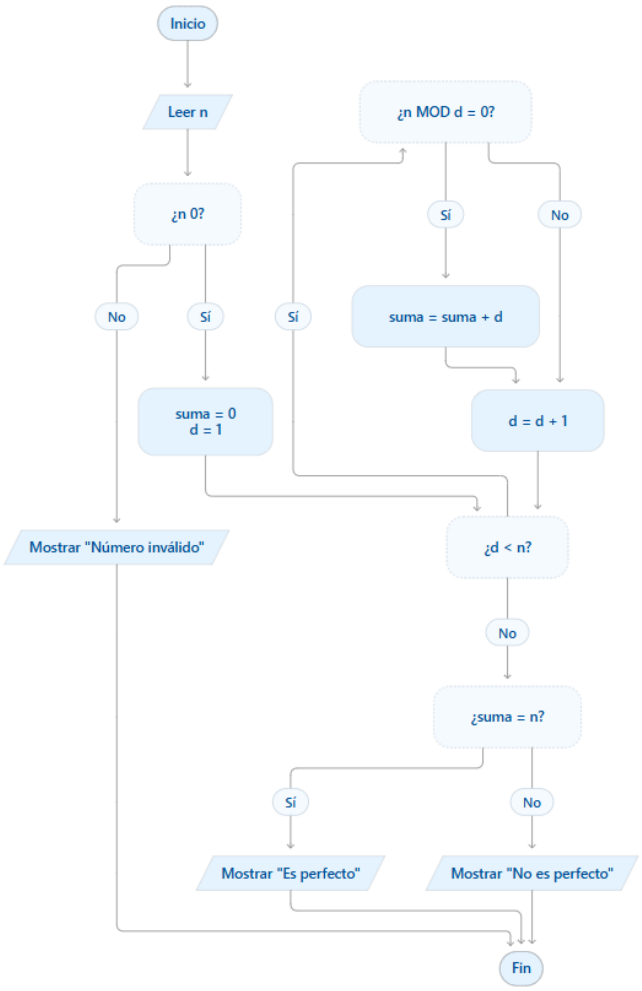
3. Seudocódigo

```
Inicio
  Leer n
  suma <- 0

  Para d <- 1 Hasta n - 1 Hacer
    Si n MOD d = 0 Entonces
      suma <- suma + d
    FinSi
  FinPara

  Si suma = n Entonces
    Escribir "Es un numero perfecto"
  SiNo
    Escribir "No es un numero perfecto"
  FinSi
Fin
```

4. Diagrama de flujo



5. Prueba de escritorio

d	¿28 % d = 0?	suma después de comprobar
1	Si	1
2	Si	3
3	No	3
4	Si	7
5	No	7
6	No	7
7	Si	14
8	No	14
9	No	14
10	No	14
11	No	14
12	No	14
13	No	14
14	Si	28
15 al 27	No	28

6. Código en C

```
#include <stdio.h>

int main() {
   int n, d, suma = 0;

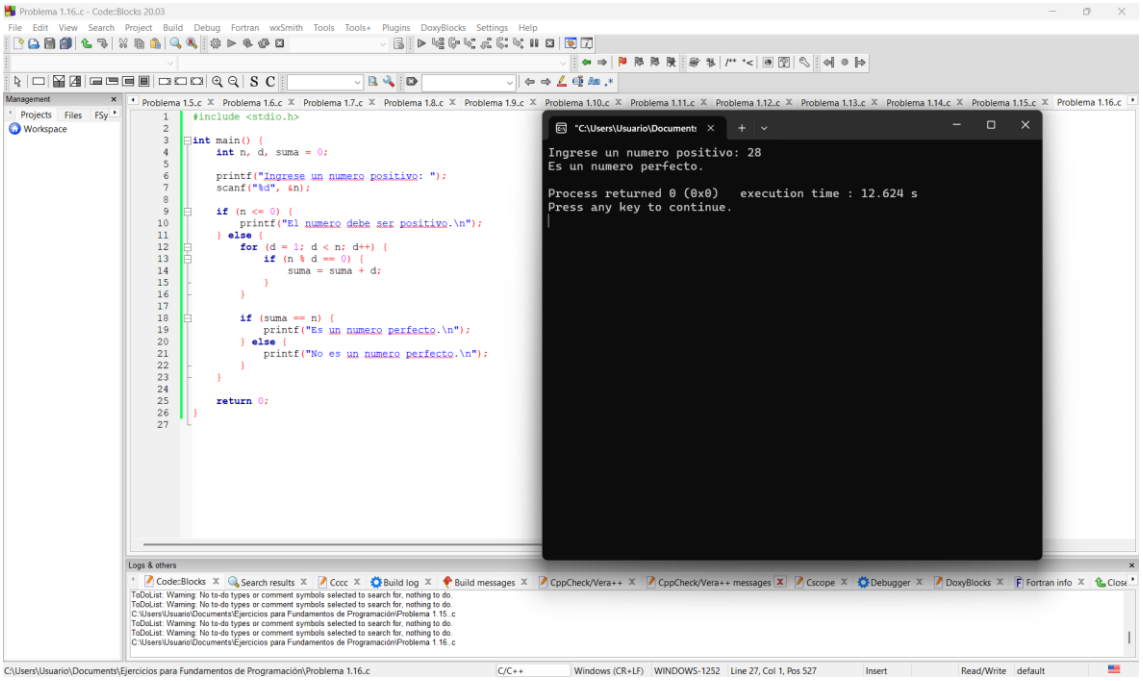
   printf("Ingrese un numero positivo: ");
   scanf("%d", &n);

   if (n <= 0) {
      printf("El numero debe ser positivo.\n");
   } else {
      for (d = 1; d < n; d++) {
         if (n % d == 0) {
            suma = suma + d;
         }
      }

      if (suma == n) {
         printf("Es un numero perfecto.\n");
      } else {
         printf("No es un numero perfecto.\n");
      }
   }

   return 0;
}
```

7. **Evidencia**



Problema 1.17. Conversión de decimal a binario

Desarrolla un programa que lea un entero positivo en base 10 y muestre su representación en binario.

1. Análisis del problema

Entrada: un número entero positivo en base decimal.

Proceso: dividir sucesivamente entre 2, obtener los restos y construir el número binario en el orden correcto.

Salida: representación binaria del número ingresado.

2. Tabla de objetos

Objeto	Tipo de dato	Descripción
n	Entero	Número decimal ingresado.
binario	Entero largo	Acumula los dígitos binarios.
resto	Entero	Resto de dividir entre 2.
posición	Entero largo	Determina la posición de cada dígito.

3. Seudocódigo

Inicio

Leer n

binario <- 0

posicion <- 1

Mientras n > 0 Hacer

resto <- n MOD 2

binario <- binario + resto * posicion

posicion <- posicion * 10

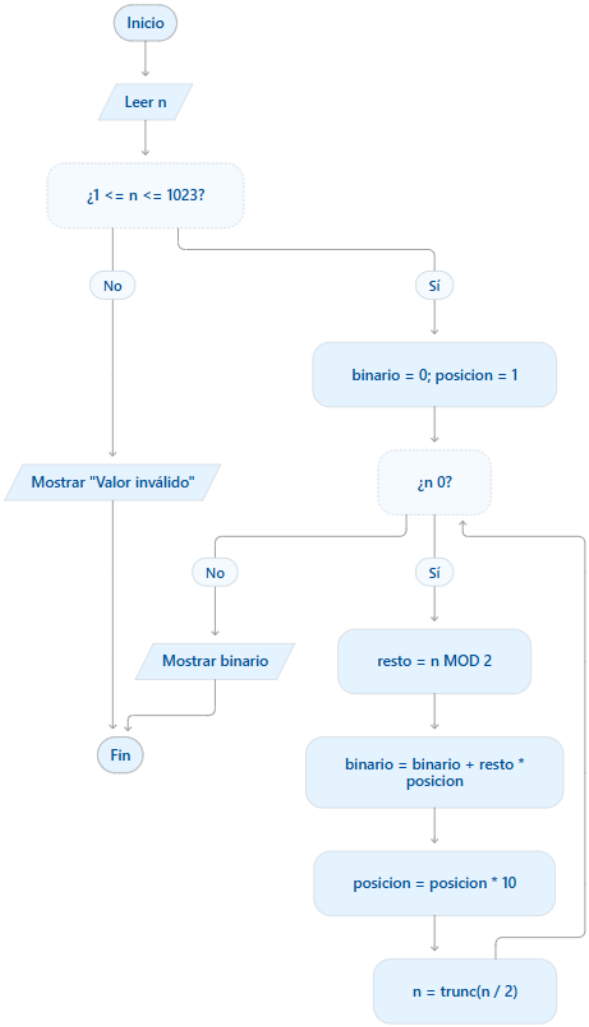
n <- trunc(n / 2)

FinMientras

Escribir binario

Fin

4. Diagrama de flujo



5. Prueba de escritorio

Iteración	n inicial	resto	posición usada	binario	n final

1	13	1	1	1	6
2	6	0	10	1	3
3	3	1	100	101	1
4	1	1	1000	1101	0

6. **Código en C**

```
#include <stdio.h>

int main() {
   int n, resto;
   unsigned long long binario = 0;
   unsigned long long posicion = 1;

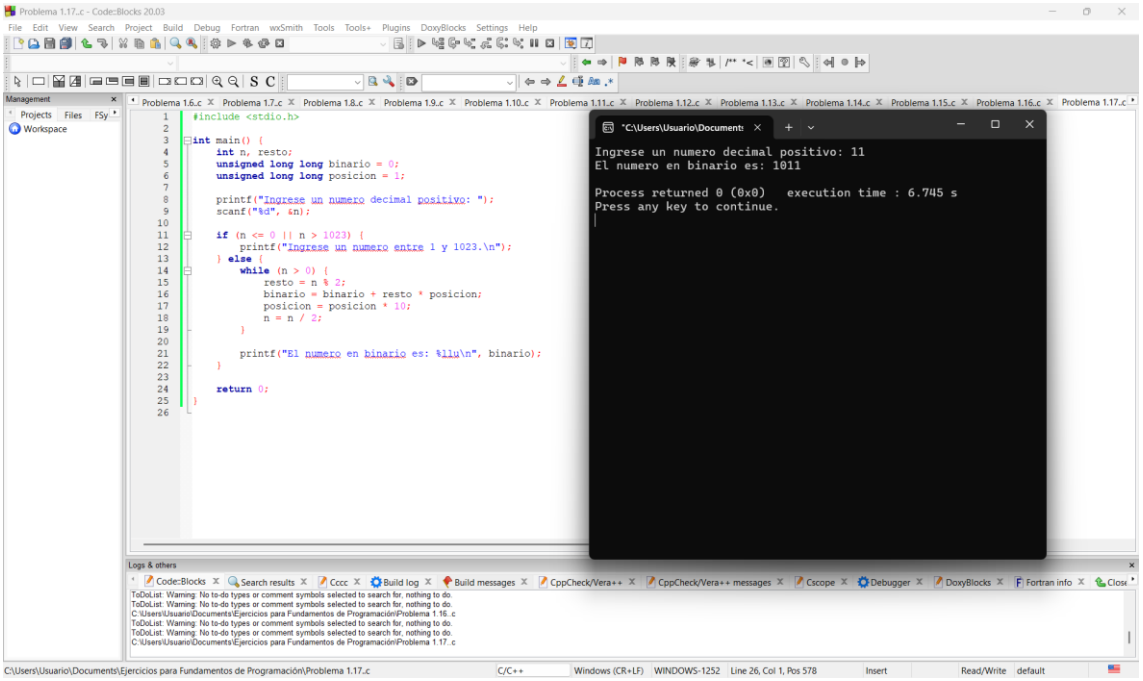
   printf("Ingrese un numero decimal positivo: ");
   scanf("%d", &n);

   if (n <= 0 || n > 1023) {
      printf("Ingrese un numero entre 1 y 1023.\n");
   } else {
      while (n > 0) {
         resto = n % 2;
         binario = binario + resto * posicion;
         posicion = posicion * 10;
         n = n / 2;
      }

      printf("El numero en binario es: %llu\n", binario);
   }

   return 0;
}
```

7. **Evidencia**



Problema 1.18. Triángulo de asteriscos

Desarrolla un programa que lea un entero n ≥ 1 y dibuje por pantalla un triángulo rectángulo de n filas, donde la fila i tiene i asteriscos.

1. **Análisis del problema**

Entrada: un número entero n mayor o igual a 1.

Proceso: utilizar dos ciclos anidados. El primero controla las filas y el segundo imprime la cantidad de asteriscos correspondiente.

Salida: un triángulo rectángulo de n filas.

2. **Tabla de objetos**

Objeto	Tipo de dato	Descripción
n	Entero	Número de filas del triángulo.
i	Entero	Contador de filas.
j	Entero	Contador de asteriscos.

3. **Seudocódigo**

Inicio

Leer n

Para i <- 1 Hasta n Hacer

Para j <- 1 Hasta i Hacer

Escribir "*" SinSaltolinea

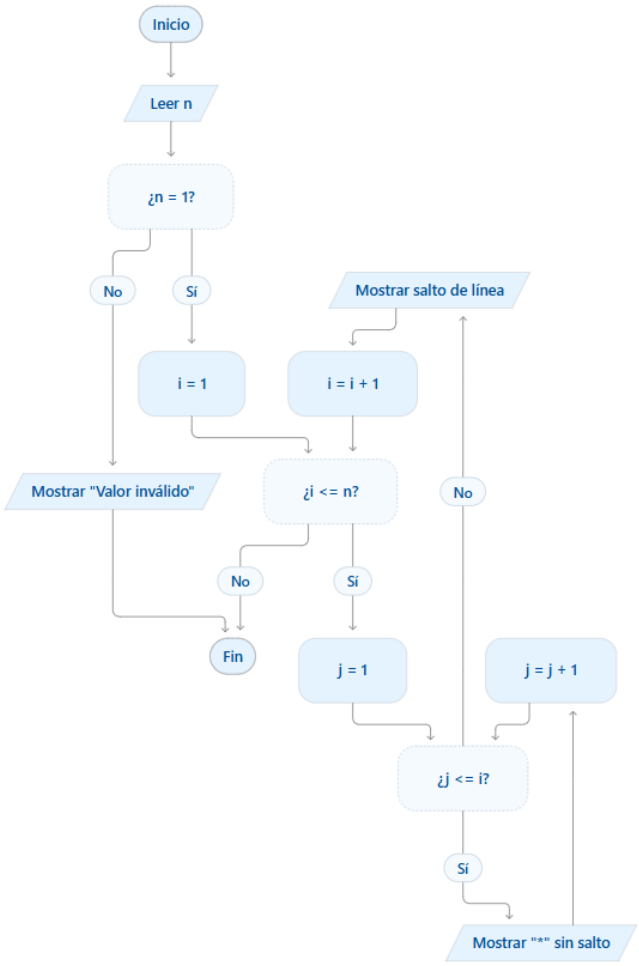
FinPara

Escribir SaltoDeLinea

FinPara

Fin

4. Diagrama de flujo



5. Prueba de escritorio

Iteración	Valores de j	Asteriscos impresos
1	1	*
2	1, 2	**
3	1, 2, 3	***
4	1, 2, 3, 4	****
5	1, 2, 3, 4, 5	*****

6. Código en C

```
#include <stdio.h>
```

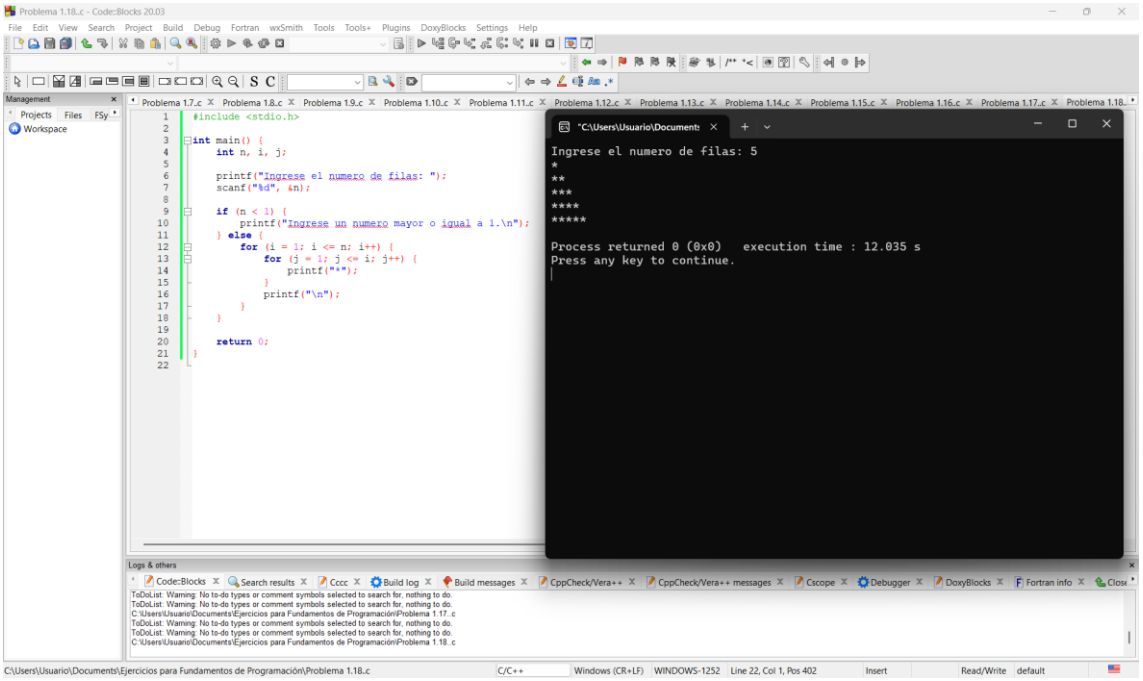
```
int main() {
   int n, i, j;

   printf("Ingrese el numero de filas: ");
   scanf("%d", &n);

   if (n < 1) {
      printf("Ingrese un numero mayor o igual a 1.\n");
   } else {
      for (i = 1; i <= n; i++) {
         for (j = 1; j <= i; j++) {
            printf("*");
         }
         printf("\n");
      }
   }

   return 0;
}
```

7. **Evidencia**



Problema 1.19. Segundo mayor de una secuencia

Desarrolla un programa que lea números enteros positivos hasta que se introduzca un 0, y muestre el mayor y el segundo mayor de los valores ingresados. Si hay menos de dos valores, debe informarlo.

1. **Análisis del problema**

Entrada: números enteros positivos; el ingreso termina cuando se introduce 0.

Proceso: comparar cada número con los valores almacenados en mayor y segundo, actualizándolos cuando corresponda.

Salida: mostrar el mayor y el segundo mayor, o informar si se ingresaron menos de dos valores.

2. **Tabla de objetos**

Objeto	Tipo de dato	Descripción
num	Entero	Número ingresado.
mayor	Entero	Mayor valor encontrado.
segundo	Entero	Segundo mayor valor encontrado.
cantidad	Entero	Cantidad de números positivos ingresados.

3. **Seudocódigo**

Inicio

mayor <- 0

segundo <- 0

```
  cantidad <- 0

  Leer num

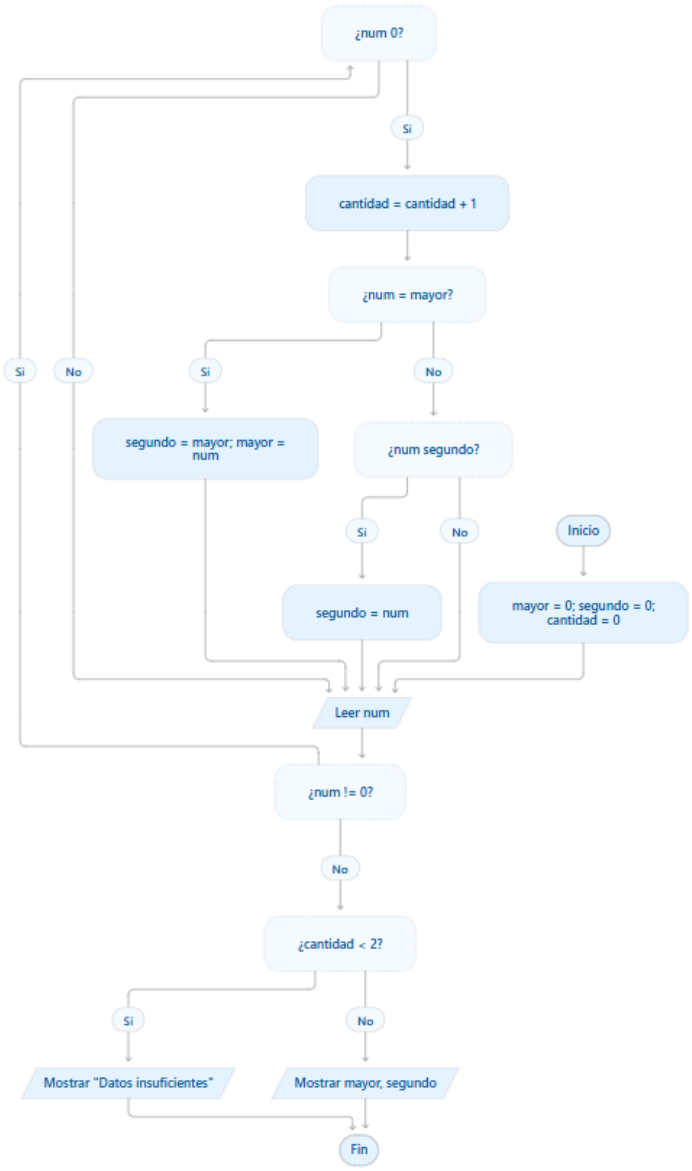
  Mientras num <> 0 Hacer
    Si num > 0 Entonces
      cantidad <- cantidad + 1

      Si num >= mayor Entonces
        segundo <- mayor
        mayor <- num
      SiNo
        Si num > segundo Entonces
          segundo <- num
        FinSi
      FinSi
    FinSi

    Leer num
  FinMientras

  Si cantidad < 2 Entonces
    Escribir "Datos insuficientes"
  SiNo
    Escribir mayor, segundo
  FinSi
Fin
```

4. **Diagrama de flujo**



5. Prueba de escritorio

Iteración	num	mayor	segundo	cantidad
Inicial	-	0	0	0
1	7	7	0	1
2	3	7	3	2
3	9	9	7	3
4	5	9	7	4
5	0	Termina	Termina	4

6. Código en C

```
#include <stdio.h>


int main() {
```

```
int num, mayor = 0, segundo = 0;
int cantidad = 0;

printf("Ingrese numeros positivos (0 para terminar):\n");
scanf("%d", &num);

while (num != 0) {
   if (num > 0) {
      cantidad++;

      if (num >= mayor) {
         segundo = mayor;
         mayor = num;
      } else if (num > segundo) {
         segundo = num;
      }
   } else {
      printf("Se ignoran los numeros negativos.\n");
   }

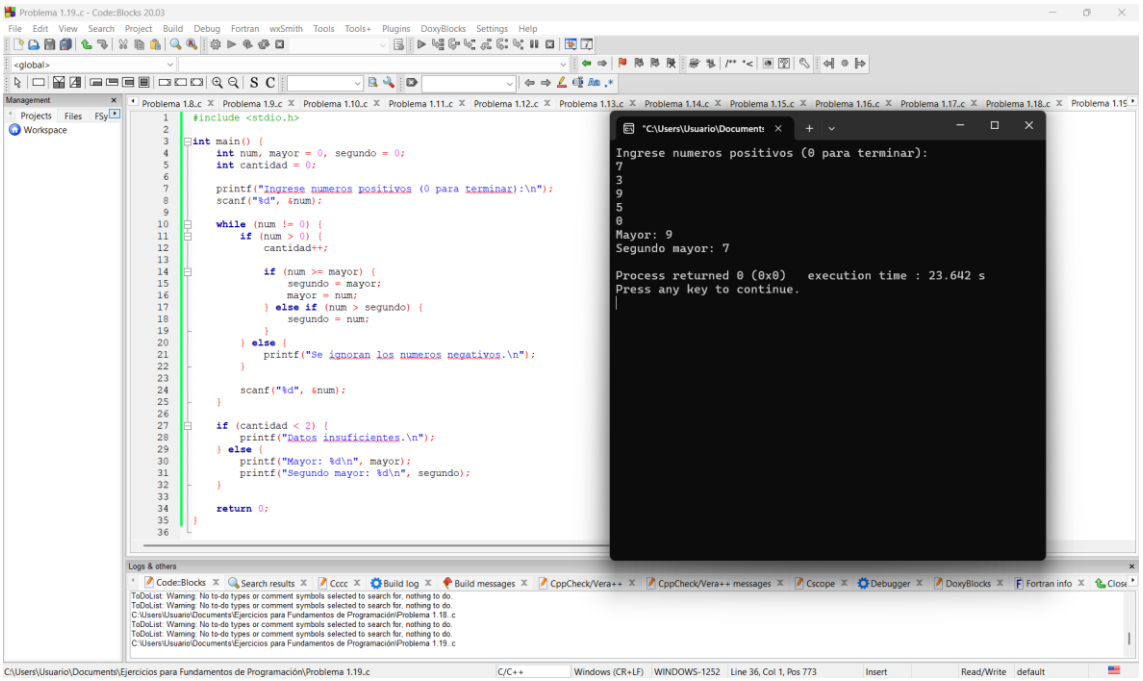
   scanf("%d", &num);
}

if (cantidad < 2) {
   printf("Datos insuficientes.\n");
} else {
   printf("Mayor: %d\n", mayor);
   printf("Segundo mayor: %d\n", segundo);
}

return 0;
```

}

7. Evidencia



Problema 1.20. Adivina el número

Desarrolla un programa que “piense” un número entre 1 y 100 (puede ser fijo para la práctica) y pida al usuario adivinarlo. Después de cada intento debe indicar “más alto”, “más bajo” o “¡acertaste!”, y al final mostrar cuántos intentos se usaron.

1. Análisis del problema

Entrada: intentos del usuario para adivinar un número entre 1 y 100.

Proceso: comparar cada intento con el número secreto e incrementar el contador.

Salida: indicar si debe intentar con un número más alto o más bajo, y mostrar los intentos utilizados cuando acierte.

2. Tabla de objetos

Objeto	Tipo de dato	Descripción
secreto	Entero	Número que se debe adivinar.
intento	Entero	Número ingresado por el usuario.
contador	Entero	Cantidad de intentos realizados.

3. Seudocódigo

Inicio

secreto <- 42

```
  contador <- 0

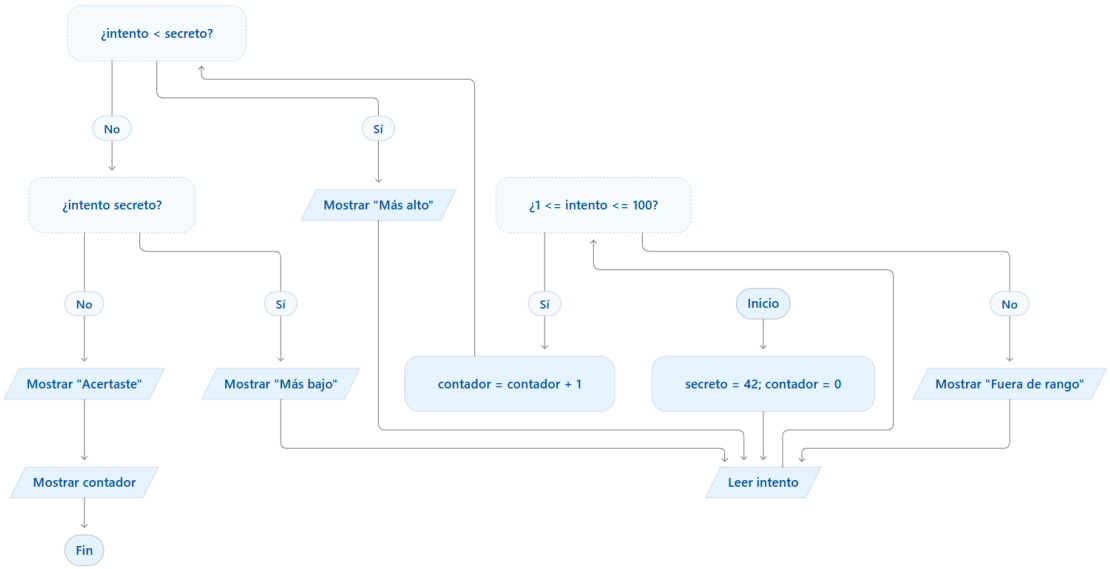
  Repetir
    Leer intento

    Si intento >= 1 Y intento <= 100 Entonces
      contador <- contador + 1

      Si intento < secreto Entonces
        Escribir "Mas alto"
      SiNo
        Si intento > secreto Entonces
          Escribir "Mas bajo"
        SiNo
          Escribir "Acertaste"
        FinSi
      FinSi
    SiNo
      Escribir "Numero fuera de rango"
    FinSi
  Hasta Que intento = secreto

  Escribir contador
Fin
```

4. **Diagrama de flujo**



5. Prueba de escritorio

Iteración	intento	secreto	Comparación	contador	Mensaje
1	50	42	50 > 42	1	Más bajo
2	25	42	25 < 42	2	Más alto
3	40	42	40 < 42	3	Más alto
4	42	42	42 = 42	4	Acertaste

6. Código en C

```
#include <stdio.h>

int main() {
    int secreto = 42;
    int intento, contador = 0;

    printf("Adivina el numero entre 1 y 100.\n");

    do {
        printf("Ingrese su intento: ");
        scanf("%d", &intento);

        if (intento < 1 || intento > 100) {
            printf("Numero fuera de rango.\n");
```

```
            continue;
        }

        contador++;

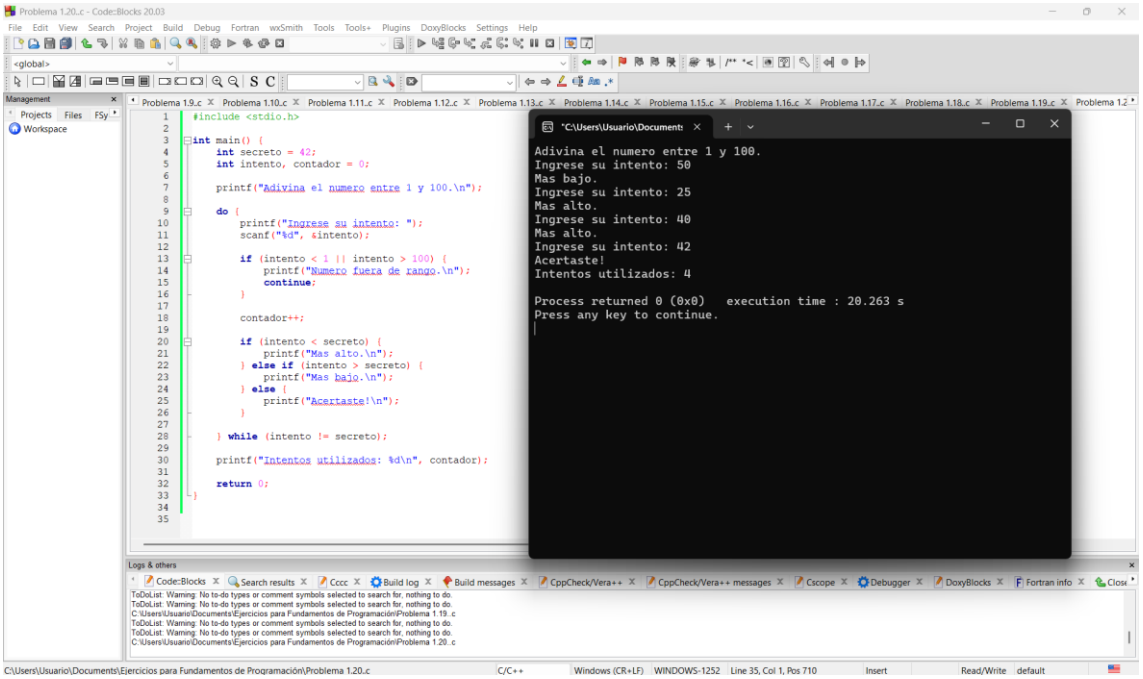
        if (intento < secreto) {
            printf("Mas alto.\n");
        } else if (intento > secreto) {
            printf("Mas bajo.\n");
        } else {
            printf("Acertaste!\n");
        }

    } while (intento != secreto);

    printf("Intentos utilizados: %d\n", contador);

    return 0;
}
```

7. **Evidencia**


Problema 1.20.c - Code::Blocks 20.03
File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help
Management
Projects Files FSy
Workspace
Problema 1.9.c Problema 1.10.c Problema 1.11.c Problema 1.12.c Problema 1.13.c Problema 1.14.c Problema 1.15.c Problema 1.16.c Problema 1.17.c Problema 1.18.c Problema 1.19.c Problema 1.2
1 #include <stdio.h>
2
3 int main() {
4 int secreto = 42;
5 int intento, contador = 0;
6
7 printf("Adivina el numero entre 1 y 100.\n");
8
9 do {
10 printf("Ingrese su intento: ");
11 scanf("%d", &intento);
12
13 if (intento < 1 || intento > 100) {
14 printf("Numero fuera de rango.\n");
15 continue;
16 }
17
18 contador++;
19
20 if (intento < secreto) {
21 printf("Mas alto.\n");
22 } else if (intento > secreto) {
23 printf("Mas bajo.\n");
24 } else {
25 printf("Acertaste!\n");
26 }
27
28 } while (intento != secreto);
29
30 printf("Intentos utilizados: %d\n", contador);
31
32 return 0;
33 }
34
35
"C:\Users\Usuario\Document
Adivina el numero entre 1 y 100.
Ingrese su intento: 50
Mas bajo.
Ingrese su intento: 25
Mas alto.
Ingrese su intento: 40
Mas alto.
Ingrese su intento: 42
Acertaste!
Intentos utilizados: 4
Process returned 0 (0x0) execution time : 20.263 s
Press any key to continue.
Logs & others
Code::Blocks Search results Cccc Build log Build messages CppCheck/Vera++ CppCheck/Vera++ messages Cscope Debugger DoxyBlocks Fortran info Close
ToDoList: Warning: No to-do types or comment symbols selected to search for, nothing to do.
ToDoList: Warning: No to-do types or comment symbols selected to search for, nothing to do.
C:\Users\Usuario\Documents\Ejercicios para Fundamentos de Programación\Problema 1.19..c
ToDoList: Warning: No to-do types or comment symbols selected to search for, nothing to do.
ToDoList: Warning: No to-do types or comment symbols selected to search for, nothing to do.
C:\Users\Usuario\Documents\Ejercicios para Fundamentos de Programación\Problema 1.20..c
C:\Users\Usuario\Documents\Ejercicios para Fundamentos de Programación\Problema 1.20.c
C/C++
Windows (CR+LF)
WINDOWS-1252
Line 35, Col 1, Pos 710
Insert
Read/Write
default